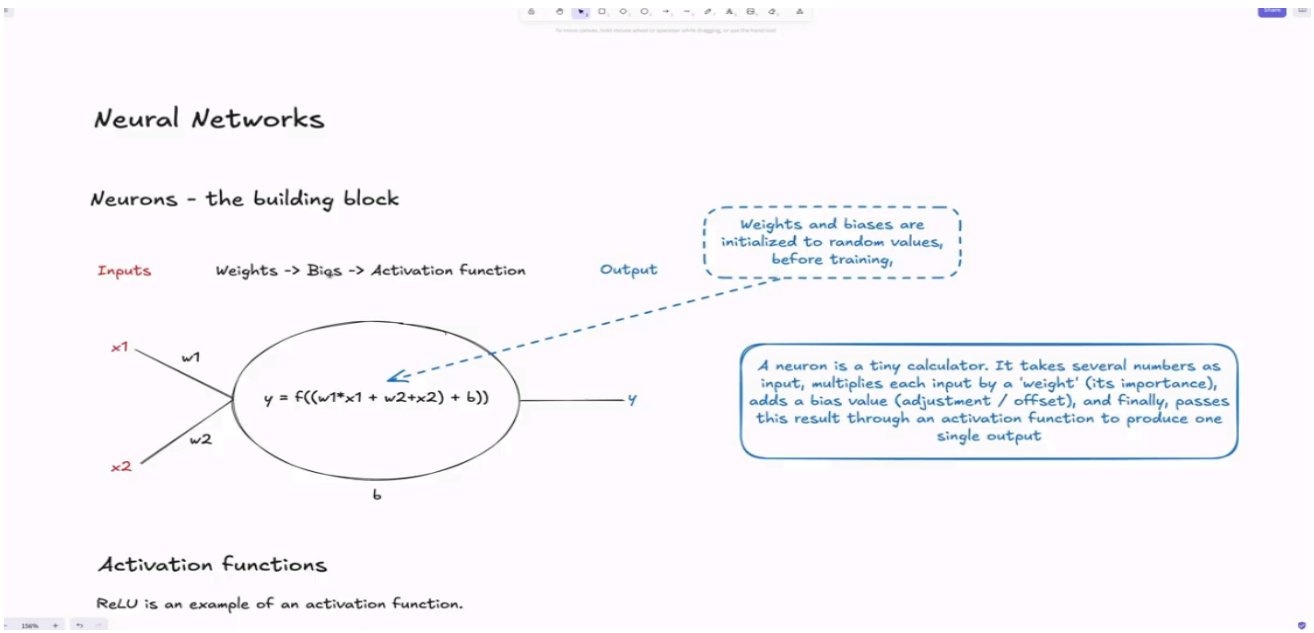
CONVOLUTIONAL NEURAL NETWORK (CNN)

Theoretical Framework & System Architecture

Neural Networks

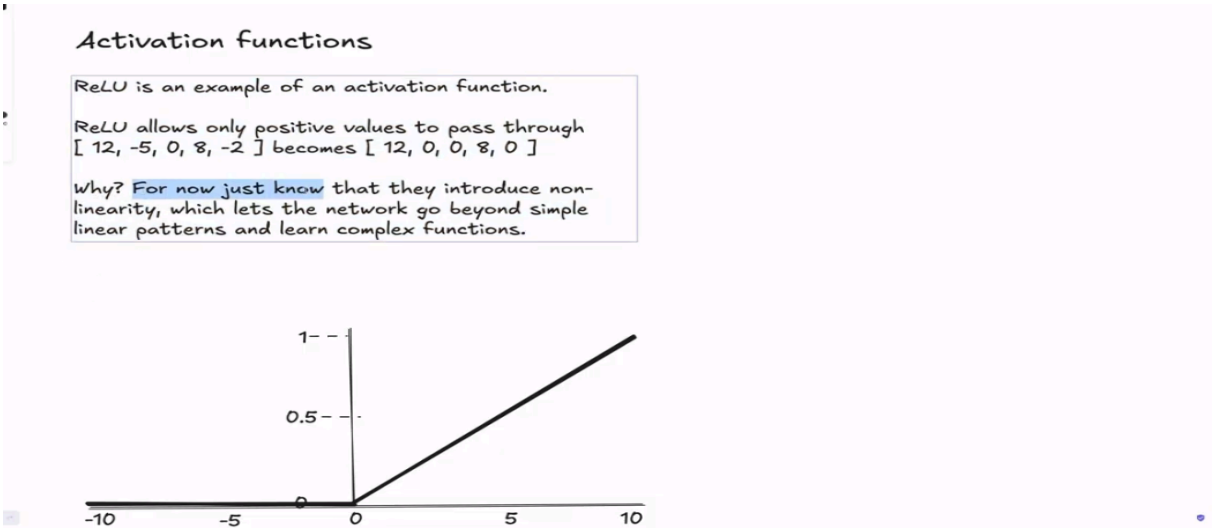
Neurons

Basic building blocks of the neural network. Several numbers as input, each input multiplied by a '**weight' (**its importance), adds a '***bias value' (adjustment / offset)*** and passes the result through an ***activation function*** to produce one single output. **Weights** and **Bias** values are initially random during training but when one trains the network they are tuned where one will expect what to see



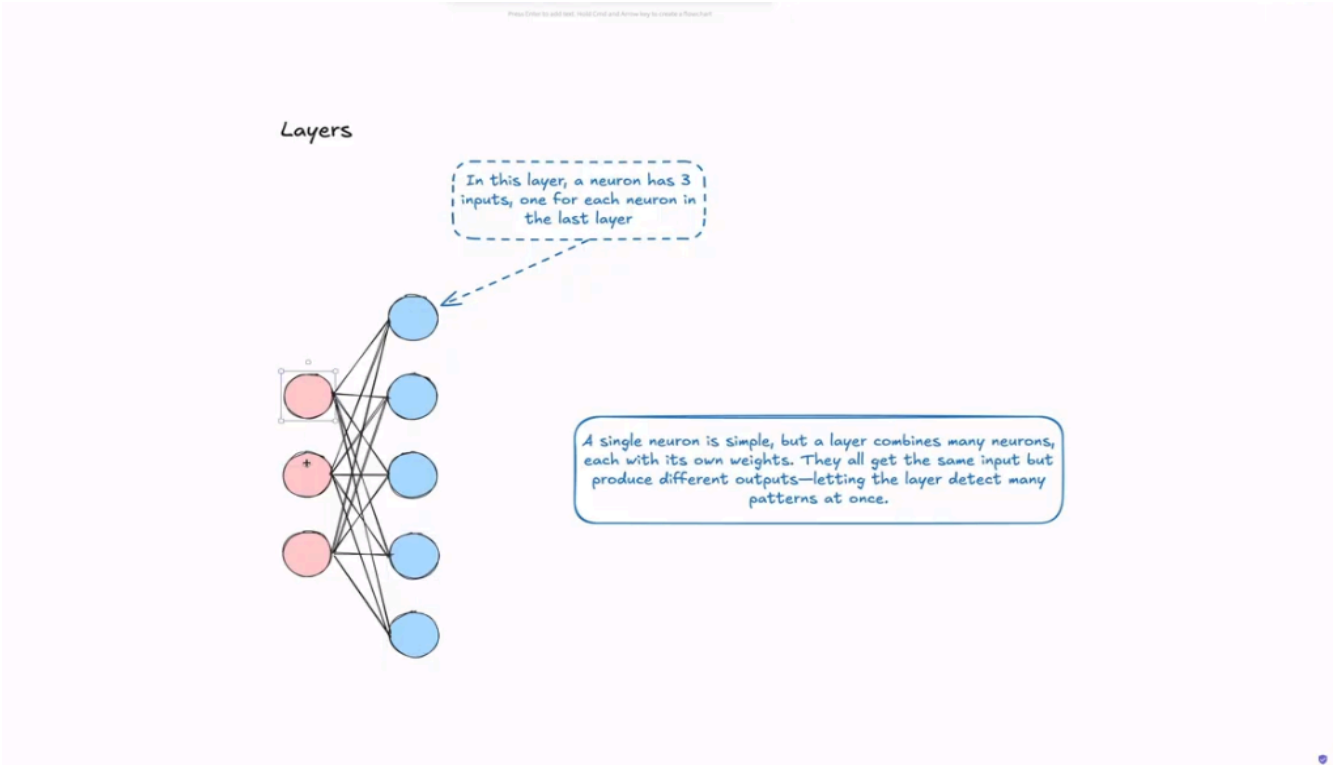
Activation Functions

They are examples of Mathematical functions e.g., **ReLU** which allows only **positive values to pass** through [13, -4, -1, 0, 2, 9] becomes [13, 0, 0, 0 2, 9]. This introduces non-linearity letting the network learn complex functions to solve real world problems (just stacking of linear functions to get complex ones)



Layers

Layer is a combination of several neurons with its weights which receive same inputs but produce several outputs.



Forward Pass

The math combines multiple neurons to form the layer. In the example below we have **inputs (**2 & 3) then the **hidden layer** (we don't care or track its weights and biases here hence hidden, we only care if the output seems correct from what we had used as inputs

-too many numbers to track) & finally the **output** neuron - returns single number based on the Math preceding it.

How it works:

Inputs	**Weights**		**Base**
2	0.3	0.2	0.4
3	0.1	0.6	0.2

From the above table the **ReLU** functions is :

relu ((2 * 0.3) + (3 * 0.1) + 0.4) = 1.3

relu ((2 * 0.2) + (3 * 0.6) + 0.2) = 2.4

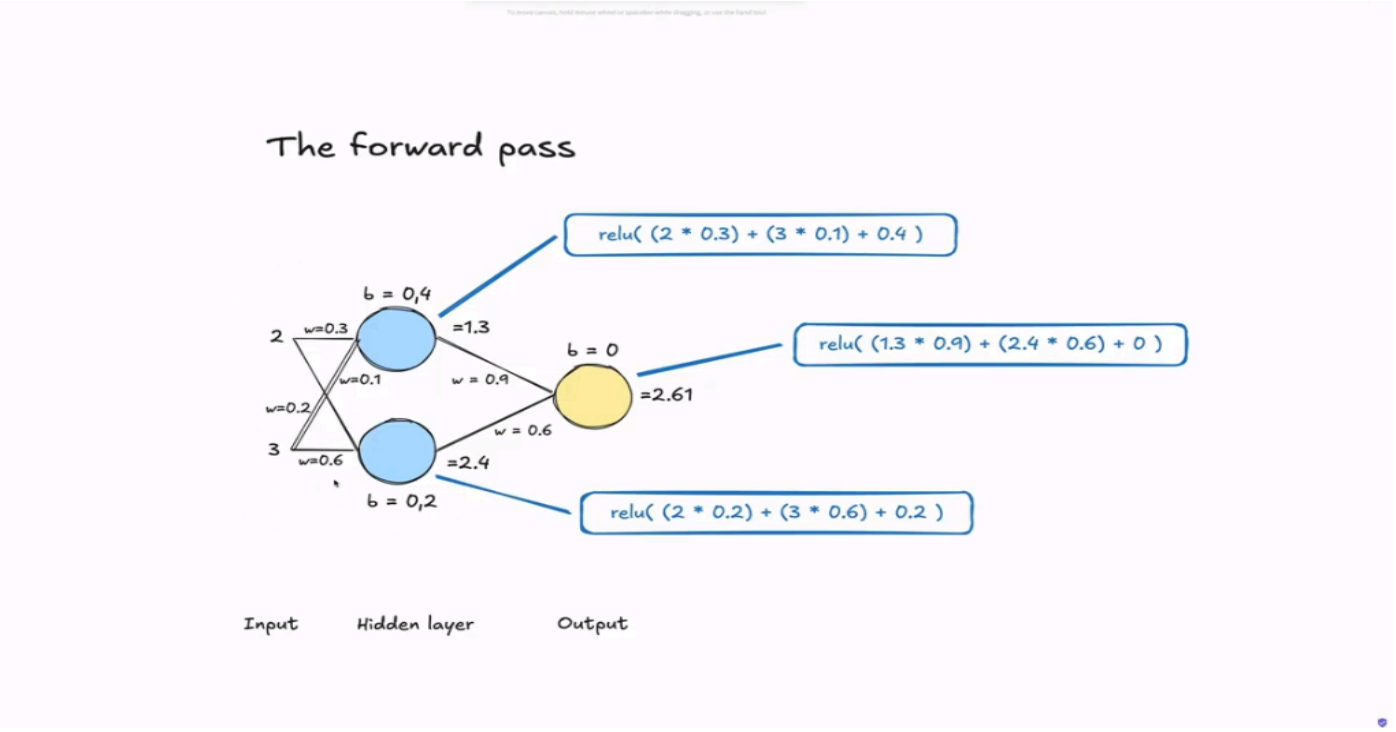
Inputs (Blue Neurons)	**Weights**	**Base**
B1 → 1.3	0.9	0
B2 → 2.4	0.6	0

From above, the **ReLU** function,

relu((1.3 * 0.9) + (2.4 * 0.6) + 0)

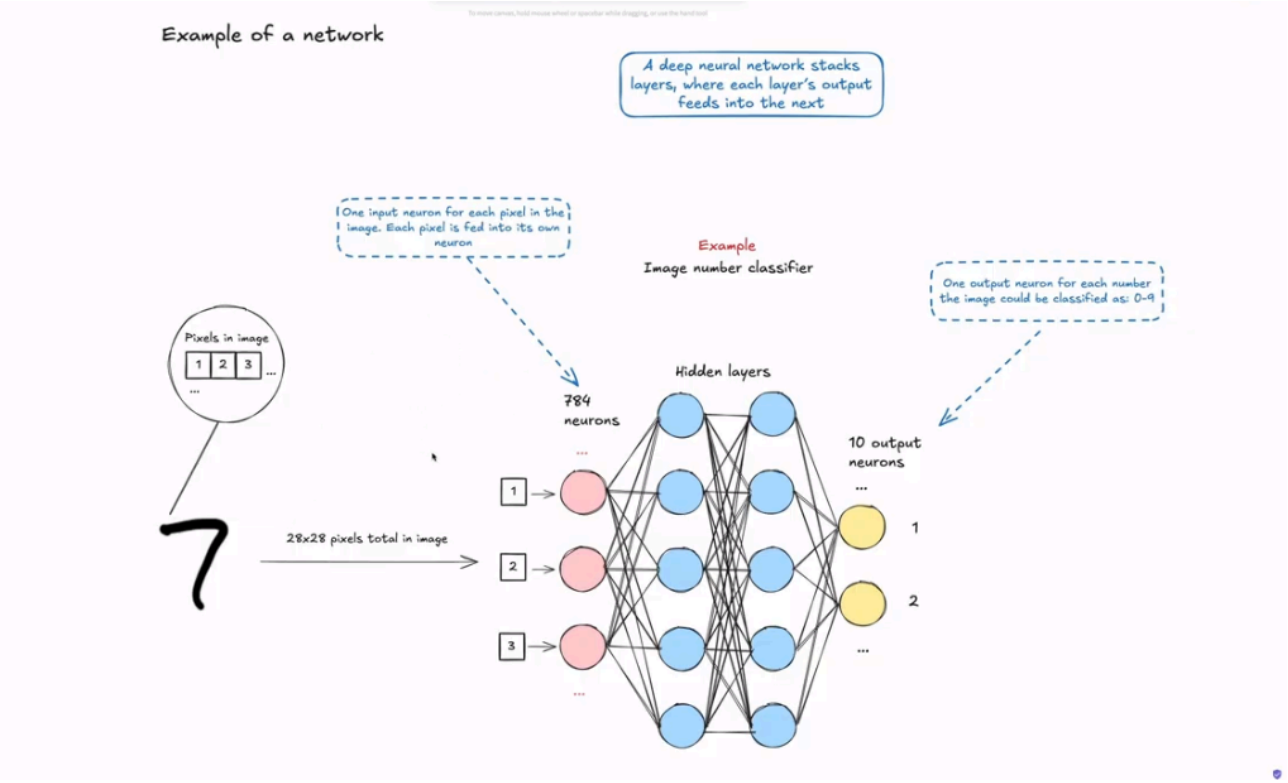
If any of the outputs we get a -ve value, relu returns 0. When you have a neural network this is what basically happens but the Mathematical computations are more complex than these and the weights and Biases are randomly set during training.

If we were to determine the prices of houses, we can set rooms as the 1st input (which we can say is more important) and 2nd as sq. foot total in the house. This will make the rooms Weight pretty high compared to sq. foot



Example of training a network:

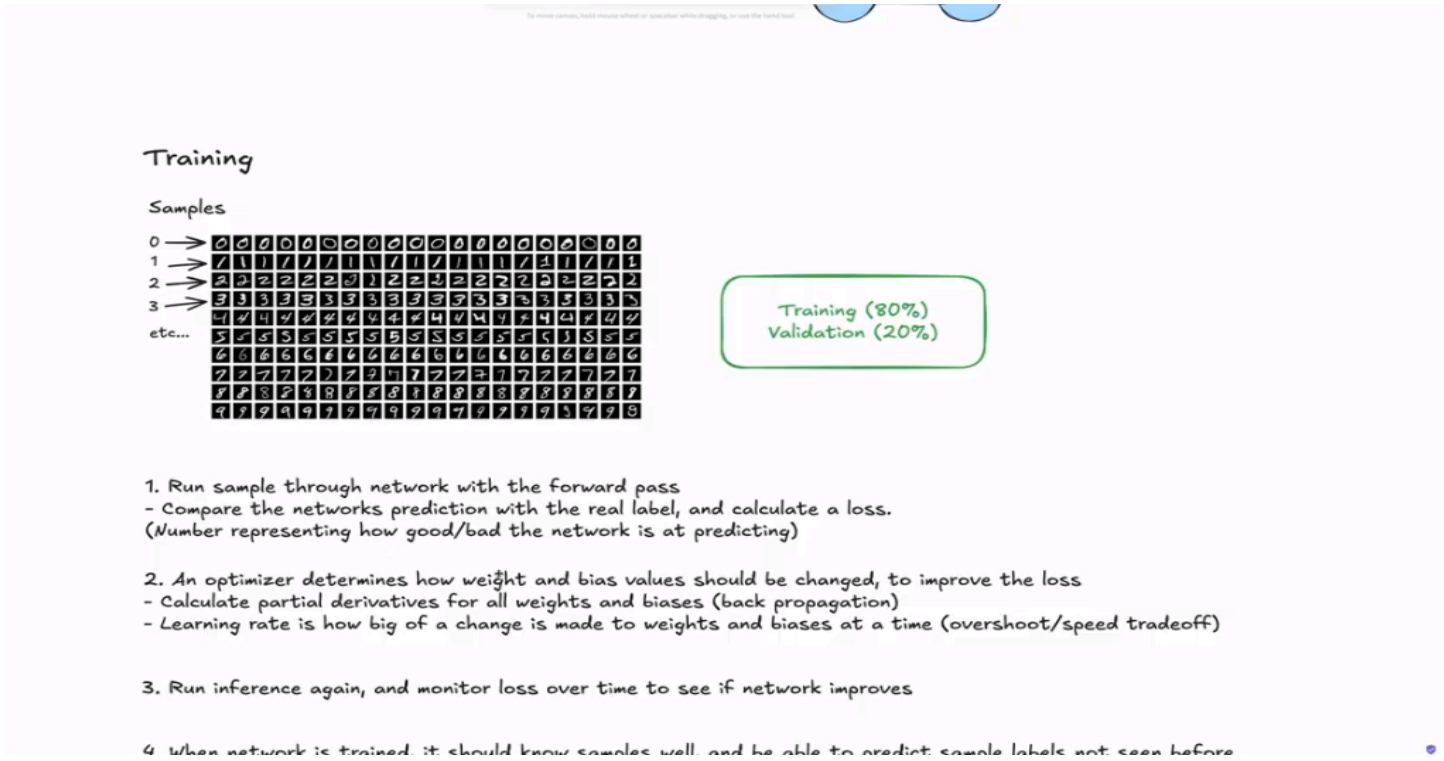
→ We want to determine how numbers will be recognized i.e., different writing styles. If one number e.g., 2 has different styles and in a picture there are **pixels (** here we use 28 * 28 pixels in image), we get a total of **784 neurons**
→ the 784 neurons become the **input layer (** its pixel its neuron, 2nd its own etc.**)**
→ In between we'll have hidden layers for complexity of the network learning and understanding the connection of the pixels (neurons) and finally the output
→ To understand how many hidden layers neurons needed is by just testing yourself or researching previous networks
→ In our case we have 0 – 10, so we will be expecting 10 output neurons out of which we now see the classification of a number. Now lets say that an image is 1 so from the output neurons, the 1 neuron will have a probability of **0.9** while that of 2 will be **0.1**



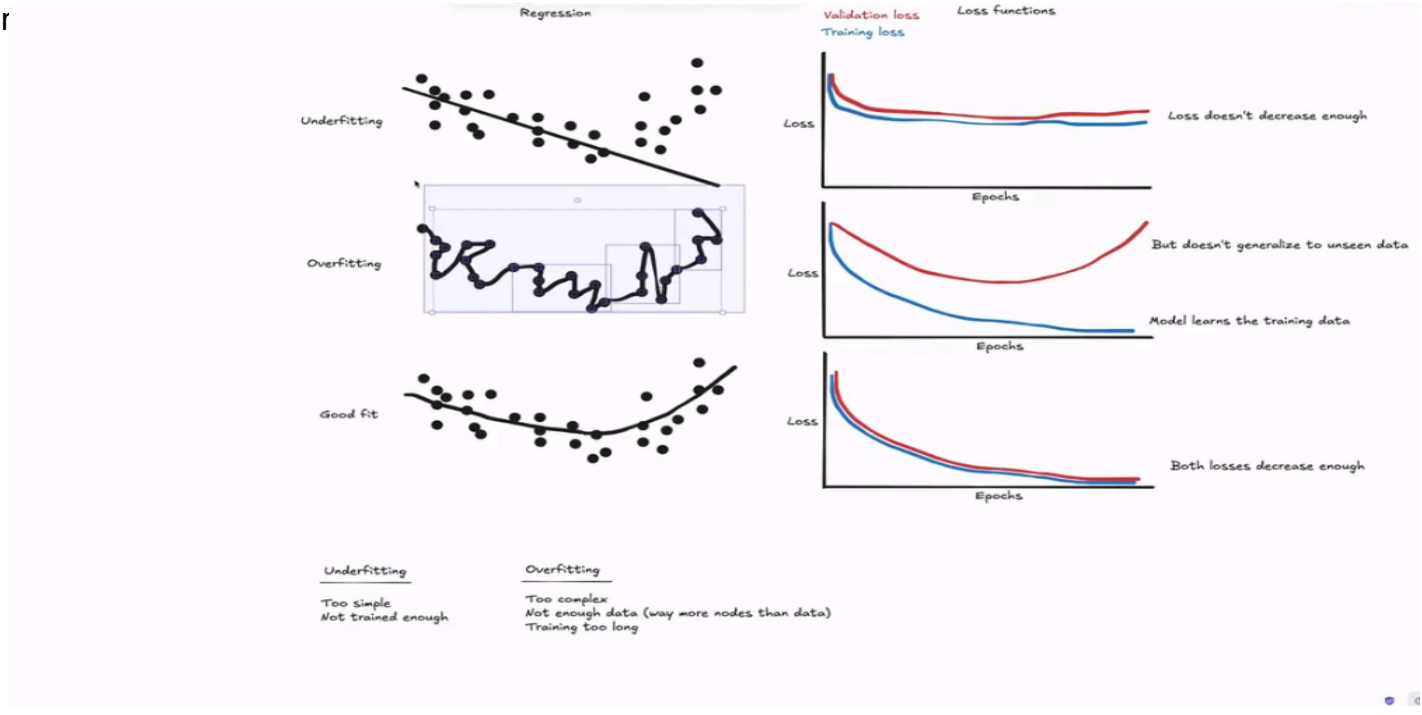
In matters training dataset:

→ **80%** is used for training and **20%** for validation. The validation part (not exactly the same as training one) is to say based on the trained, have you generally learnt anything not just memorizing the dataset

1. Sample through the network with forward pass, do a comparison of network predictions with real labels and find a **loss –** number representation of network's goodness/ badness predictability. If the number is wrong(output of network) then the loss is going to be pretty high, if the number is correct the good.
2. **Optimization –** using an **optimizer,** we determine the degree of change of the weight and bias values for loss improvement. **Back propagation** is the entire process by using partial derivative calculations for the weights and biases. From the **ReLU** functions above, getting their partial derivs. Help us know where to tune the values but not by how much. From this we get **Learning rate –** *rate of change of weight and biases (*tradeoff between **undershooting** and **speed of tradeoff)***.** *If we had a low learning rate then speed will be low because of small changes at a time, but in case of high learning rate we might have an overshoot of learning values(big increments might not know optimum values for biases and weights)*
3. Run the inference again and again while monitoring the rate of loss to see network's improvement. Here we have **Validation loss –** based on the validation dataset and T**raining loss –** based on the training dataset.
4. After network training, it should know samples and have the ability to predict previously unseen sample labels.



The following image shows some pitfalls while training models with thee line representing ability of model to predict the general trend of the models:
→ **Underfitting –** doesn't get the trend of the model. If the graphs (**loss functions)** don't go down much then the model isn't learning hence underfitting. It can be that the model is simple or not trained enough
→ O**verfitting –** hyper focused to all the points because it remembers every single point of the dataset(out of intensive training of the dataset) but validation loss increases as it doesn't remember other values other than trained ones. Might be due to too many neurons or trained data for too long.
→ **Good fit –** good enough not to remember all training dataset point but not also to ur

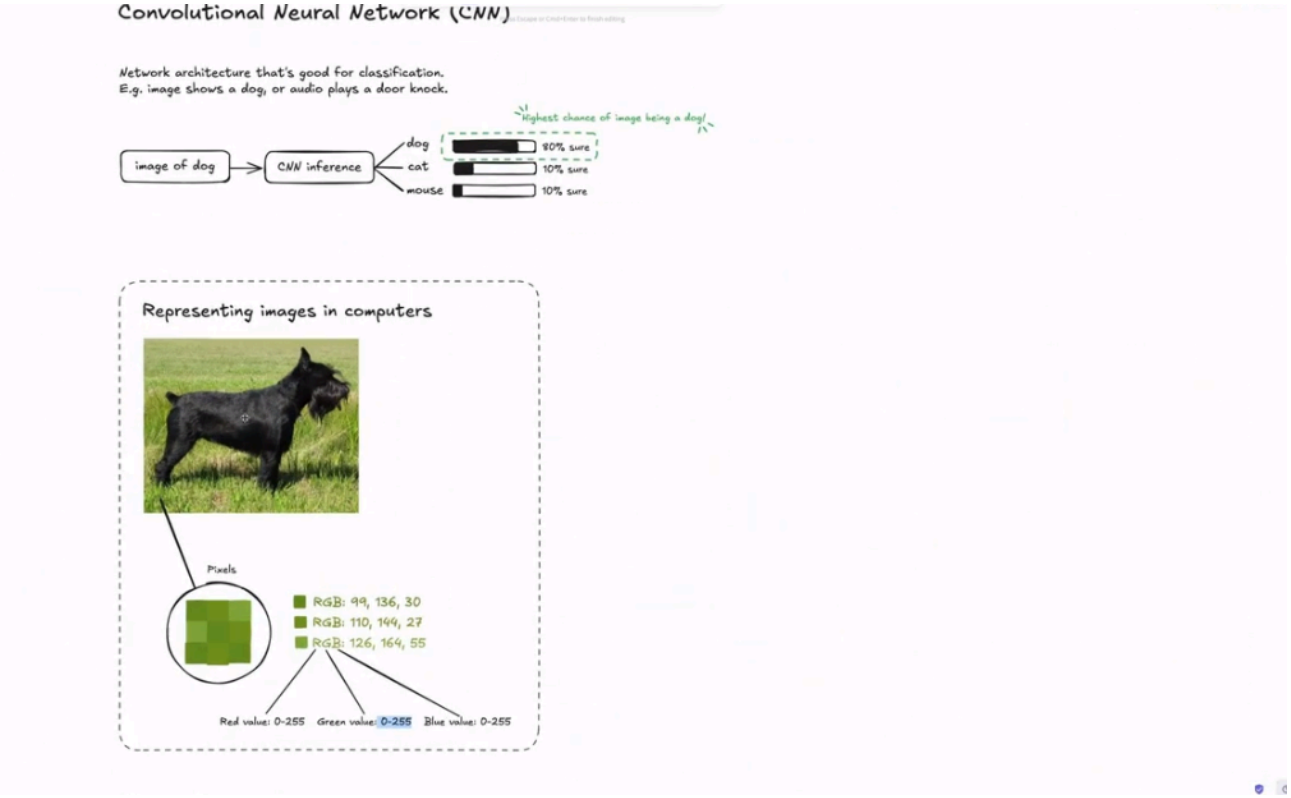


At some point of training, the loss functions tend to converge, because it doesn't really know what to learn from the dataset

Convolutional Neural Network

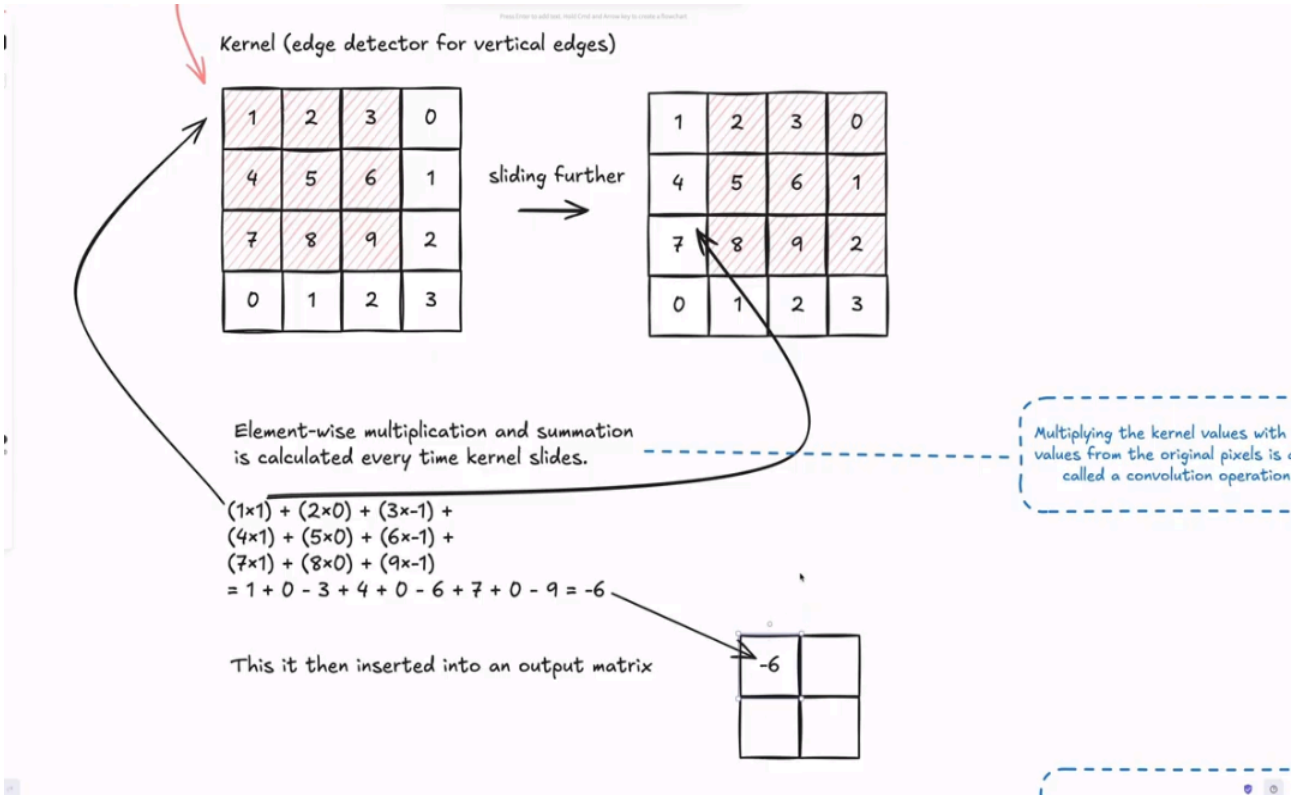
This is a way of working with a large data eg, a picture of a cat can have RBG which is complex in terms of number of pixels and neurons in matters Neural Network

CNN uses **Kernel (** edge detector for vertical edges**)** to scan across images and learn the image features instead of looking at pixels one by one. The image below with the matrix beginning with 1, 2, 3, 0 can be pixel value for Red channel of image (RGB) which makes input channel for the CNN



Kernel can slide on the input matrix and convolutional operations can be formed from the numbers. Then we can perform ***Element-wise multiplication*** and ***summation*** which is calculated every time the kernel slides and then the output of the calculations entered into an **output matrix** with the calculations shown in the pictures below for vertical edge transformation (here vertical lines of image are highlighted).

Breaking down original images into simple feature maps, CNN can learn the combination of these features into complex objects.

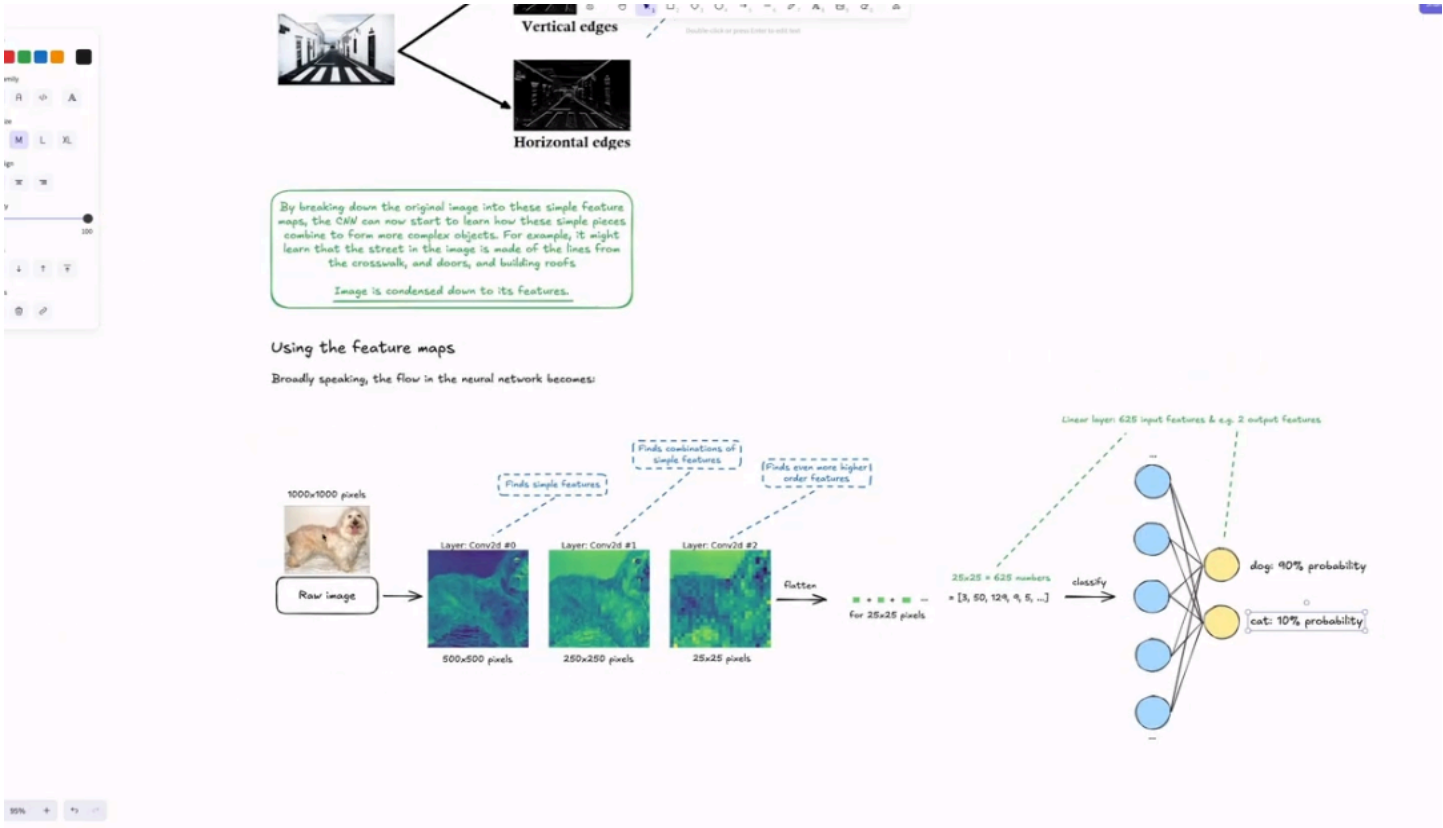


Feature Maps

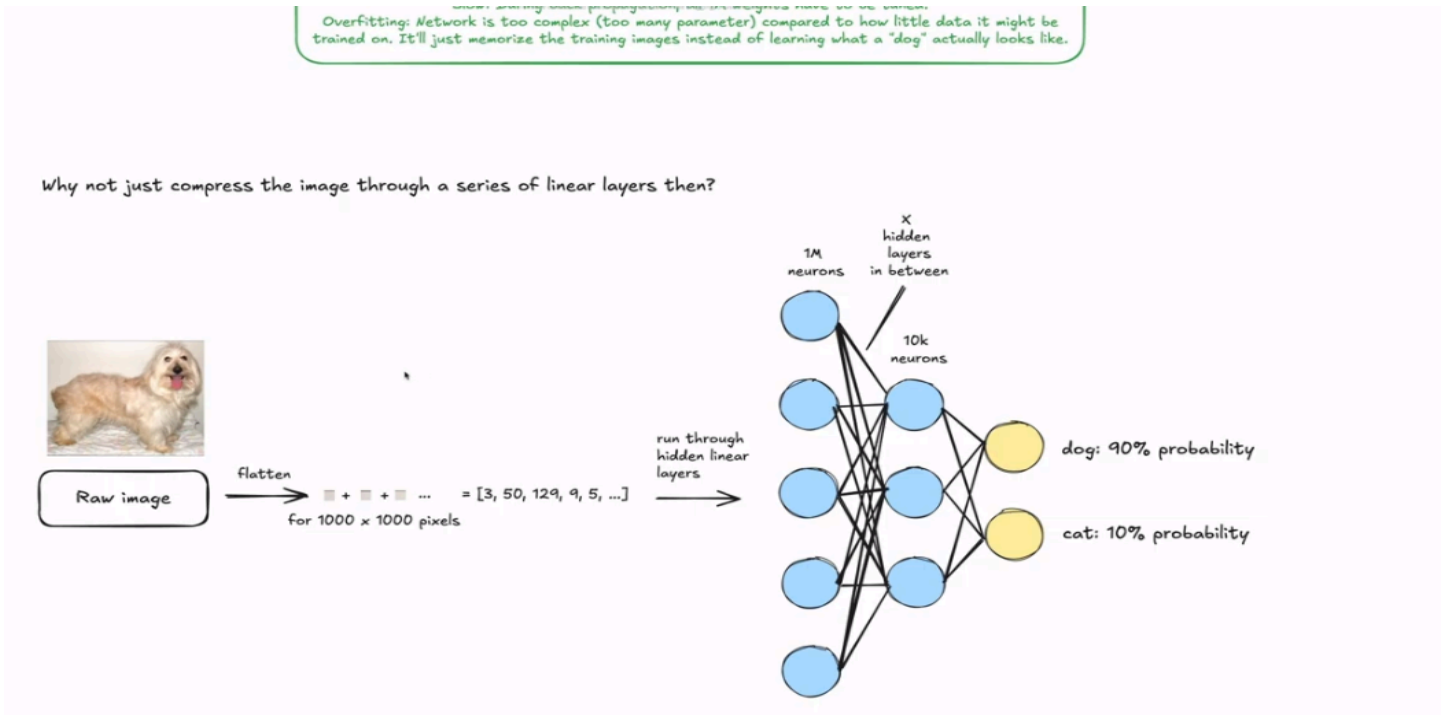
Flow of Neural Network involves a raw image passed through several Neural Networks eg a 1000 * 1000 pixel picture is split to 500 * 500 pixels (1st Network) to find simple features of dog then 250 * 250 pixels to 25 * 25 pixels to find even more Higher Order Features.

The 25 * 25 is then **flattened (** take entire image into a single list **)** which means input of **625 input features** and say 2 output features (we want to know if it's dog or cat)

So instead of the model having to learn the 1000 * 1000 pixels it only learns the important features of the picture **(** 25 * 25 pixels **).**

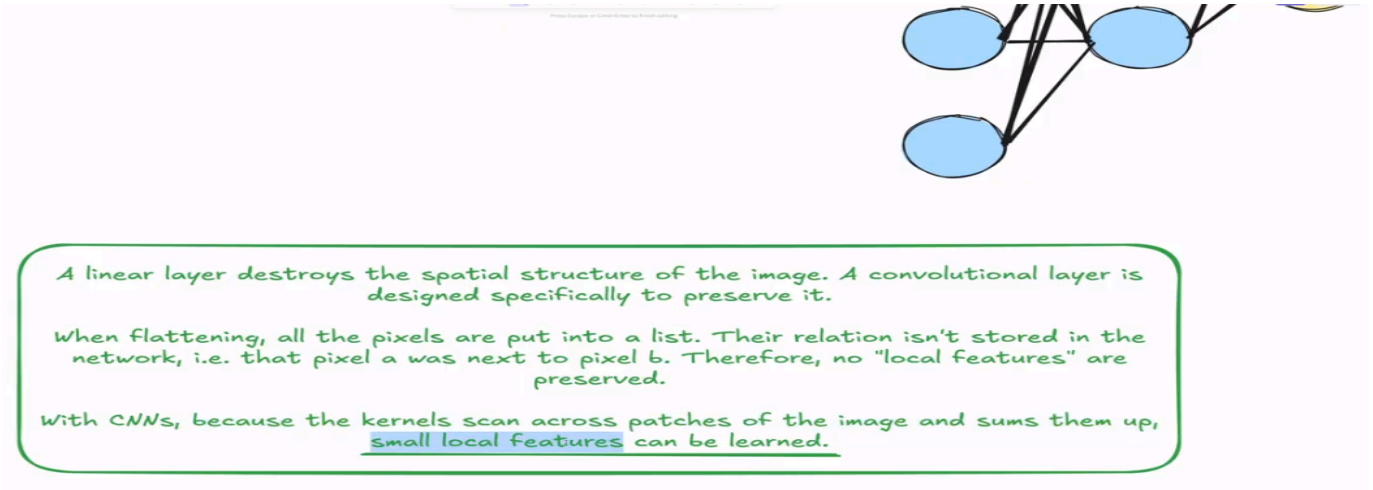


Difference with this is using **Linear layer –** instead of the convolutional layers, the image (1000 * 1000 pixels) is flattened into 1M numbers as input features (neurons).



What this means is that the network figures out the total combinations similar to image being a dog. During training, **back propagation,** all 1M weights have to be tuned which is

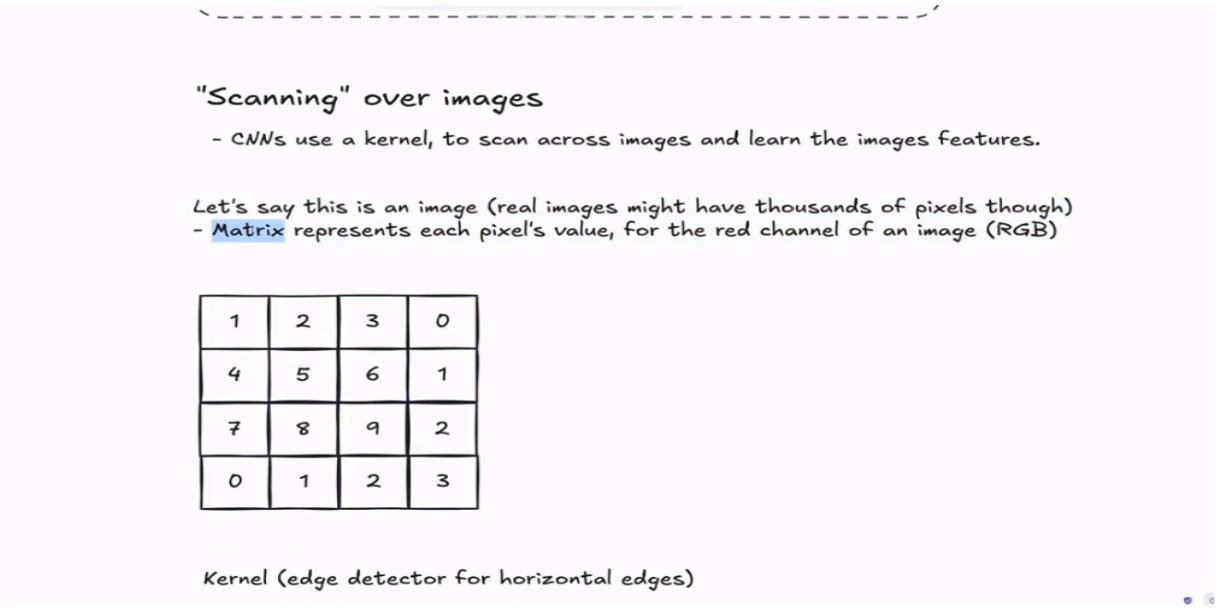
pretty slow process. Also **Overfitting** as the network is too complex (too many params) in comparison to the little data trained on.



So then why not compress the image into a series of linear layers? The reason is initially we still have the huge 1M input numbers which is enormous computationally as every neuron has its own weights and biases that have to be tuned but in the CNN the Ws & Bs tuned are in the **kernel** meaning the Ws and Bs can be reused to train the model

Another problem is **Translational Invariance. Kernel in CNN** – used to detect position of an image as it can slide over a matrix. It will then use the values to detect the position of an image, e.g., if it can be used to detect tongue it can be used severally to figure out several positions a tongue placed severally in the same image. Learning the look of an eye at top left of an image has its own Ws and Bs which are different compared to if the same tongue is placed at the bottom of the image.

The network can't use what it has learned at top to detect images at bottom. With CNN the **kernel** can slide and numbers can be tuned by CNN to extract values to make network perform as best as possible



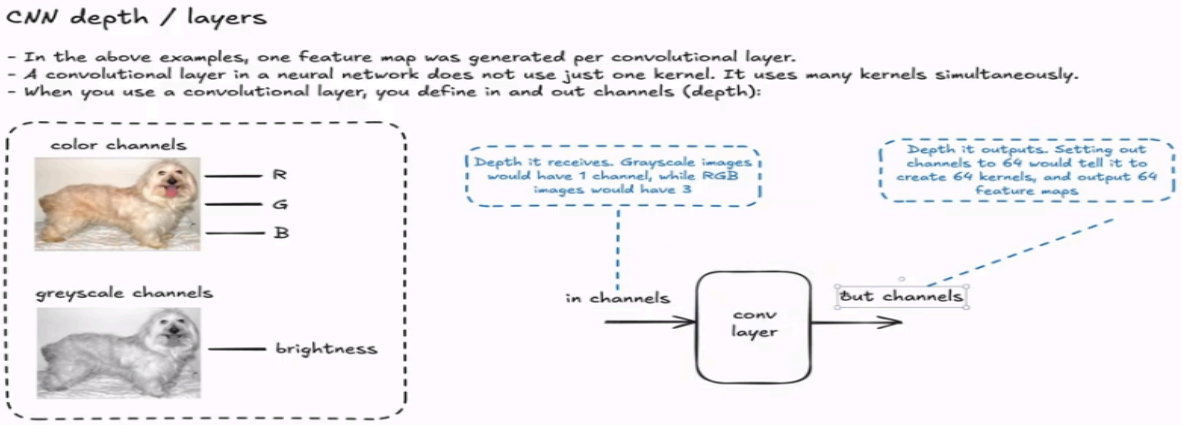
Spatial Information – relationship between pixels to determine what image is e.g., closeness of red pixels finally shows us a red rose. What flattening does is destroy this spatial information and loose meaning behind the image. **Linear Layer –** destroys spatial structure of image while **Convolutional layer –** designed to specifically preserve it.

With CNN because the Kernels scan across patches of image and sum them up, small local features can be learned.

CNN Depth / Layers

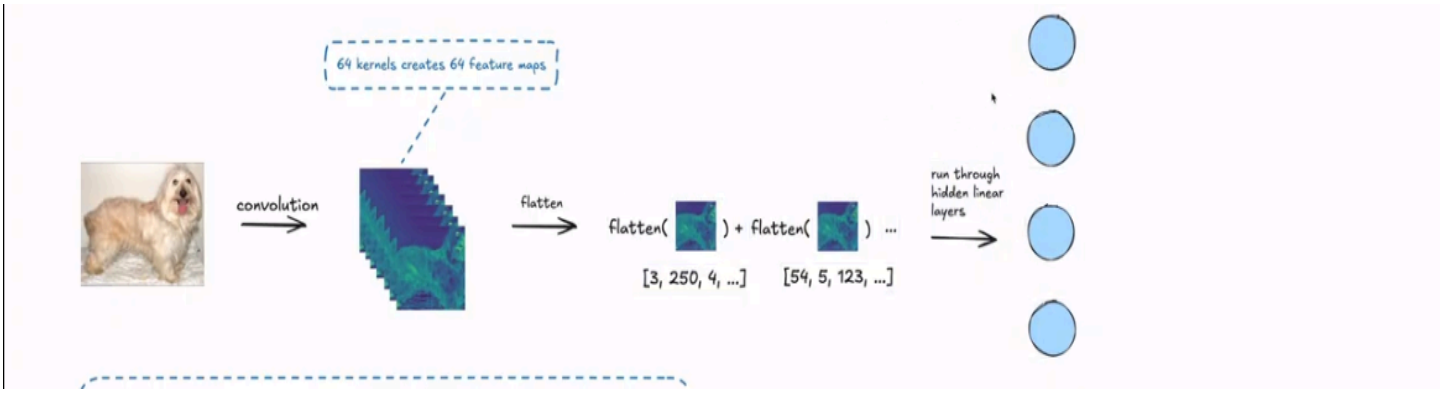
In conv layer we define **in channels (** depth it receives **)**and **out channels (** depth it outputs **).** For **in channels** its the depth it receives eg in grayscale images its 1 channel (brightness of the image) while an RGB would have 3 channels (RGB). Then **out channels** its the depth it outputs eg setting 76 channels would tell it create 76 **kernels** and 76 **feature maps.** The model will then set it with randomly initialized values and try tune kernel values to extract specific features of the image to minimize loss as much as possible (we don't know the values – done automatically by the framework).

When building CNNs, a single layer needs to be pretty powerful to be able to look into vertical or horizontal edges, specific algorithms, simple curves and combination of these and extract most important features and pass the results into the network and only way is using a lot of kernels and each tasked to find a specific feature. Then for several conv layers out channels of one needs to match the in channel of the next otherwise it won't work well.



The 76 feature maps aren't pictures anymore but rather **concept (heat) maps** of specific features and when we flatten the maps we are creating a list that summarizes all the 76 different learned features across the image. If we take a sample of the features and run

then into a series of linear layers, they won't care about exact **X-Y coordinates** of the eye only detection of the eye feature. Instead of pixel correlation it becomes abstract signals being detected ie in an image it says the fur, eye, or even tongue feature is active hence the image is a dog. But if human image is used, the fur, tongue and even eye features won't be active. The network reasons with those features.



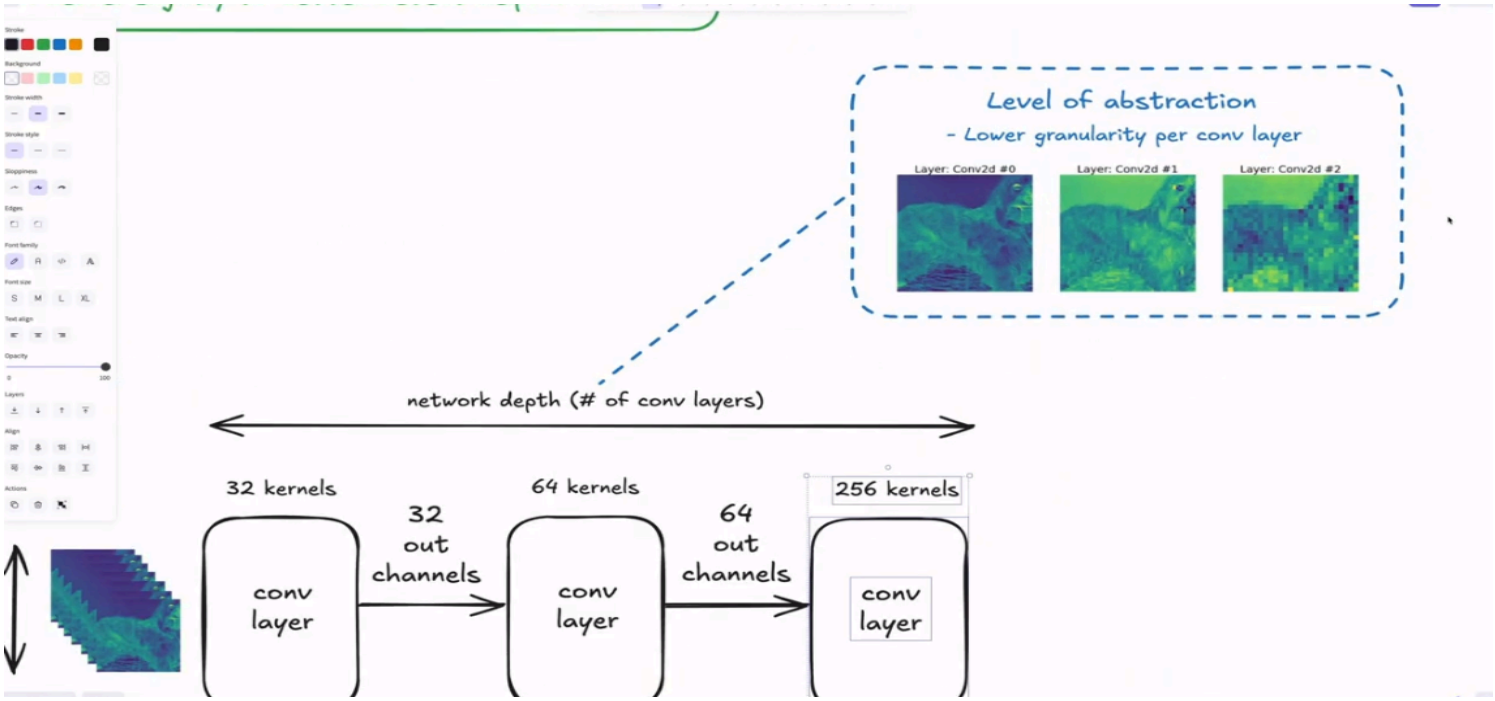
The kernels are initialized at random values to ensure that they start with slightly different values and extract slightly different features from the pictures. Also a possibility of all the kernels learning 76 unique features better than if they were all initialized with the same values. For better detection of helpful features, the network undergoes **Back-propagation a**djusting each Kernel.

Width and Depth

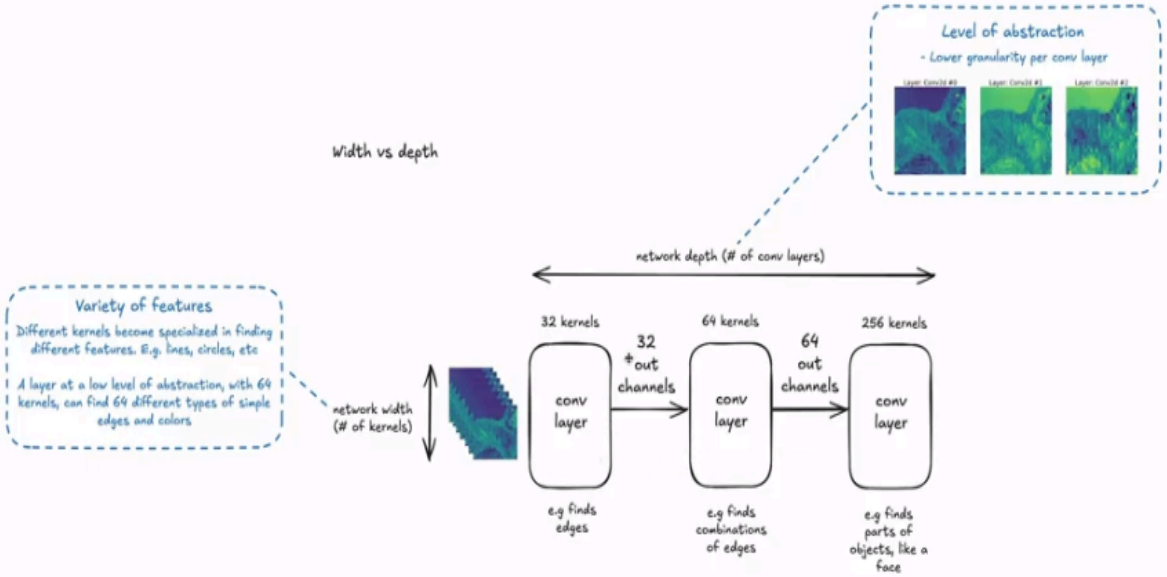
Width – talk of how many out channels in a network. It controls a variety of features because the more of these features we generate, the more of the random feature maps we generate. The kernels specialize in finding different features. A layer with 76 kernels can be able to create 76 features.

Depth – how many convolutional layers we put together (layer of abstraction adding to new layer). New layer stacked from previous one with each layer finding more complex features. We also need a large number of conv. Layers to provide a higher level of understanding.

Modern CNN have both width and depth to be able to abstract high level features such that they can make predictions in the output.



During training, the network predicts. If it's wrong, an error is sent backward (backpropagation) to adjust each kernel slightly to better detect helpful features.



Pooling, activation functions and batch norm

So far we've looked at

But often you'll see this sequence after conv layers

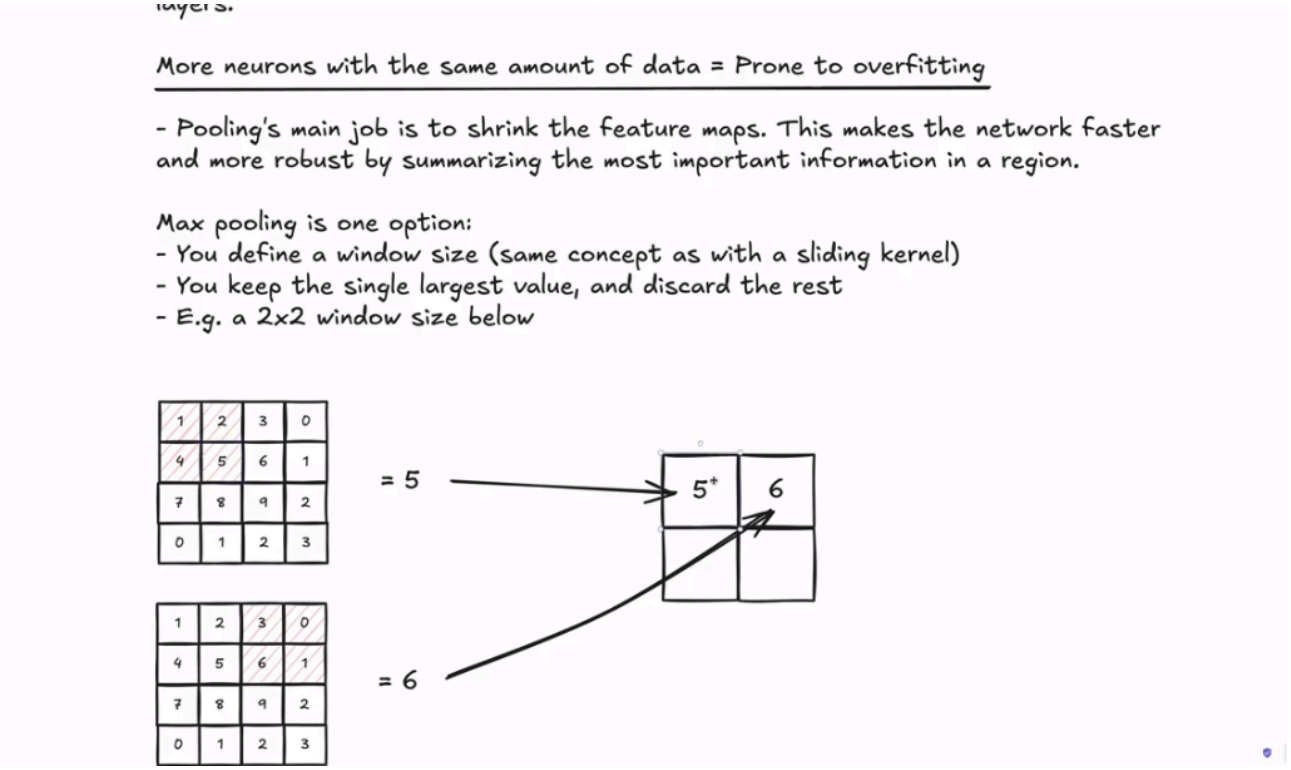
Pooling, Activation Functions and Batch Norm

Pooling layer and Activation Function are between each of conv. Layers. **Activation Function (ReLU Function)** – often used with these conv. Layers add **non-linearity** to the layers adding many simple pieces into complex functions hence non-simple linear mappings are avoided. By putting activation functions after each layer, the conv layer learns more complex things.

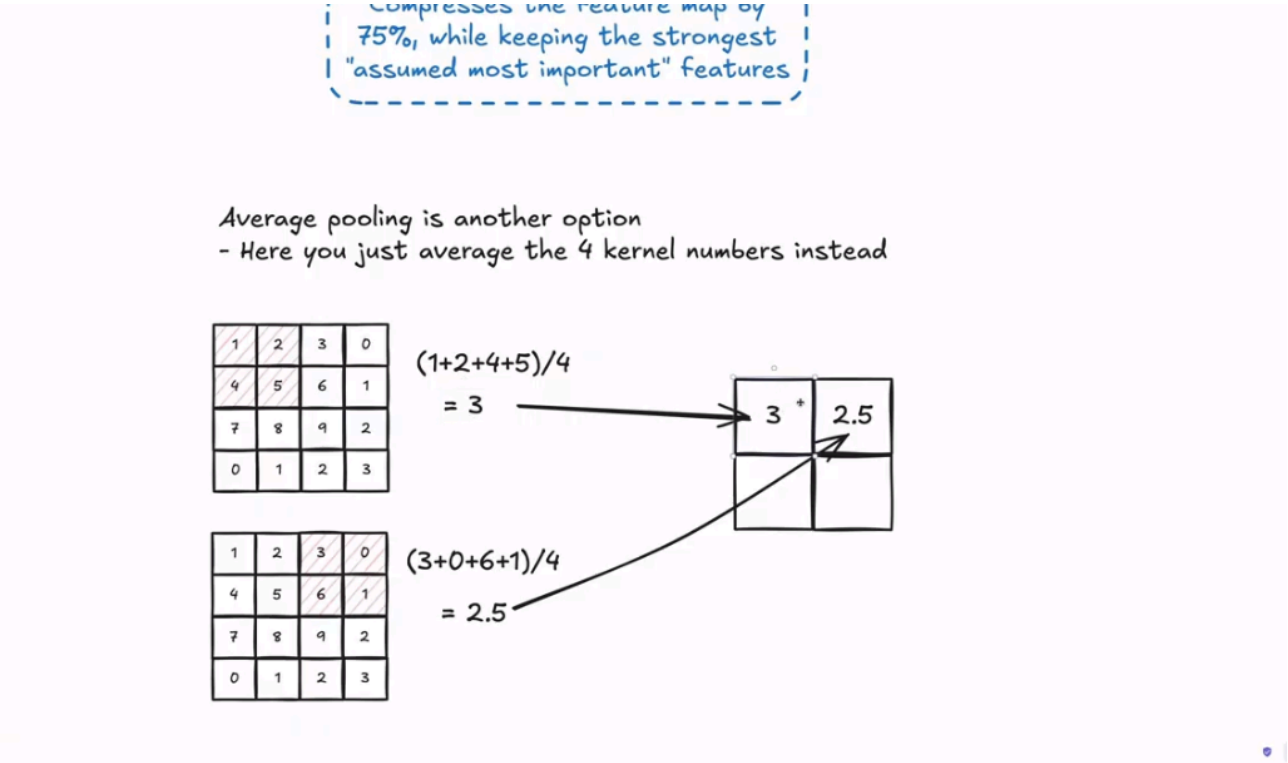
Pooling layer – the function of this is **down-sample** the feature maps so they don't have big dimensions. Conv. Layers extract features, feature maps then are created which tend to create a lot of data (huge **vectors**) leading to introduction of more features / **neurons** into the layers. If eg we have **600 * 600 pixels picture** as feature maps and we then have 76 of those then we need to flatten those into a single block of data and put that into a linear input layer.

This is a lot of input neurons (prone to **overfitting)** of the input layer. We will then be interested in reducing the width and depth of the feature maps, shrinking them, fastening the network and summarizing the most important info in a region.

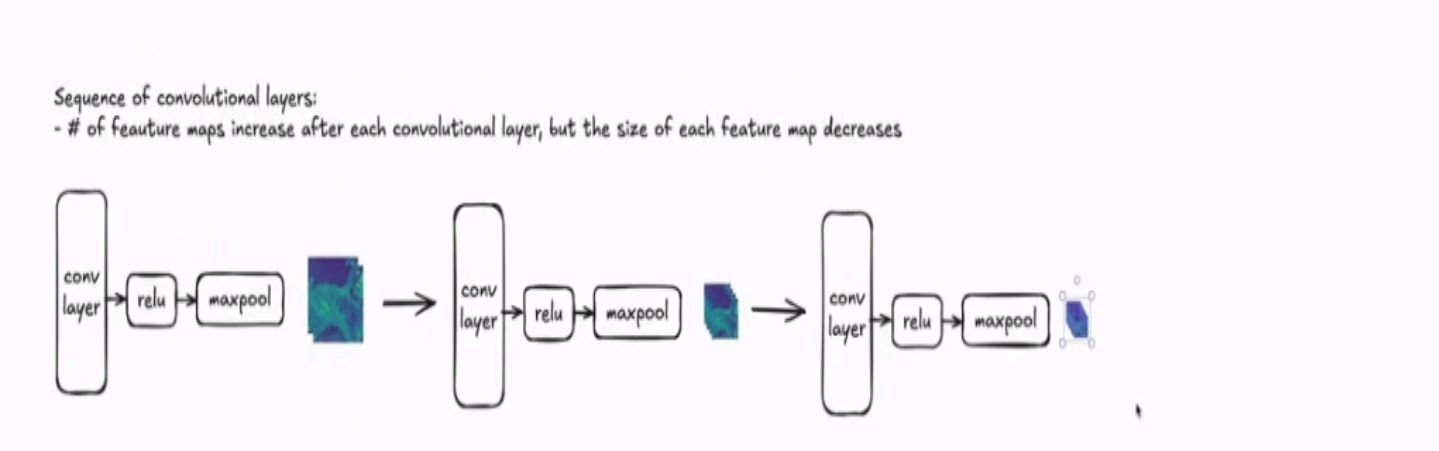
Max pooling (one option **) -** define a window size **(** same as sliding kernel **),** then keep single largest value and discard the rest eg if a matrix has a 4 * 4 we slide eg 2 * 2 matrix and pick the largest value from the slide say 5 and 6.



Average pooling – just average in eg 4 * 4 matrix the 4 kernel numbers

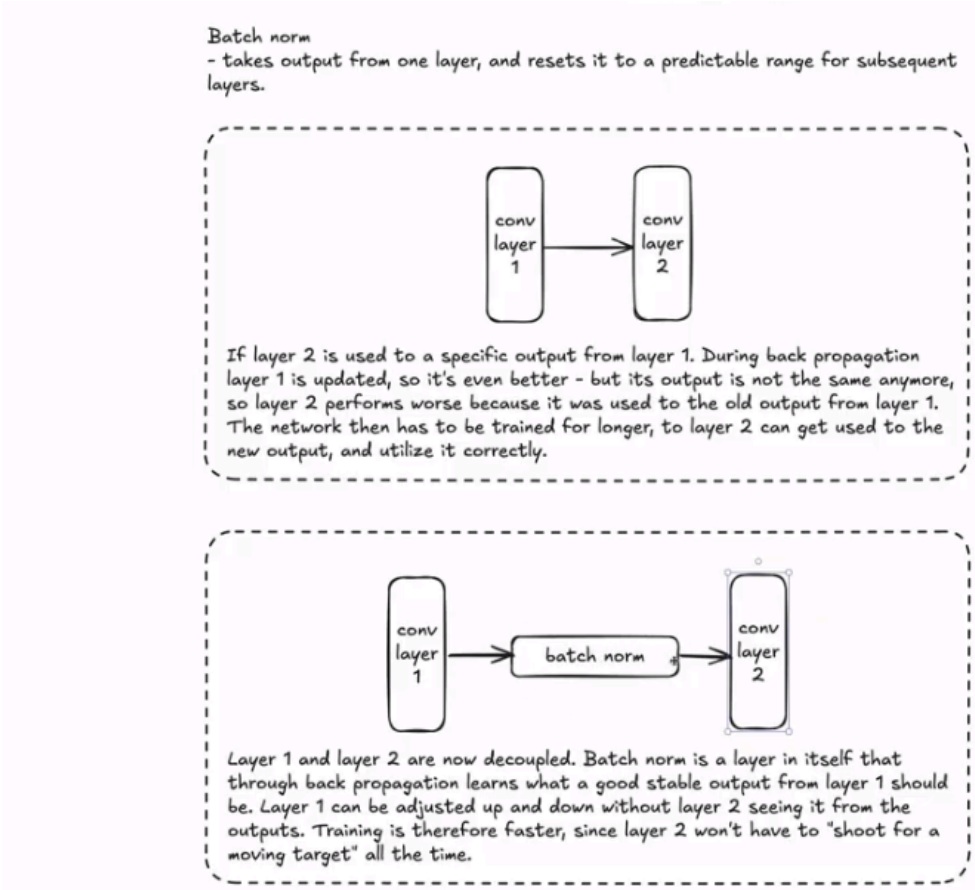


Sequence of Convolutional Layers – because of the pooling mechanism, the number of feature maps after each convolutional layer is indirectly proportional to their size. This reduces the pixels to flatten and reducing the input neurons

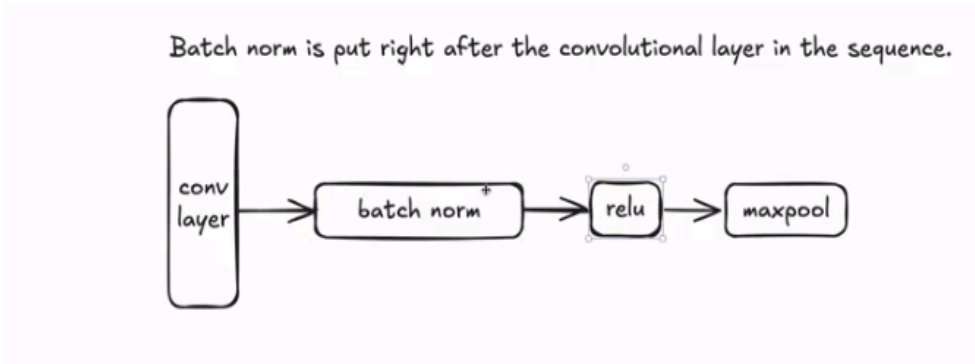


Batch Norm

Neural Network is a stack of new different layers with learning based on the output from the previous one. This may lead to some instability during training as these layers keep updating **weights & biases** eg if layer 1 has updated to layer 2 and return another new better output which L2 is still dependent on the previous one then has to update what we call **moving target** hence slow training and instability.



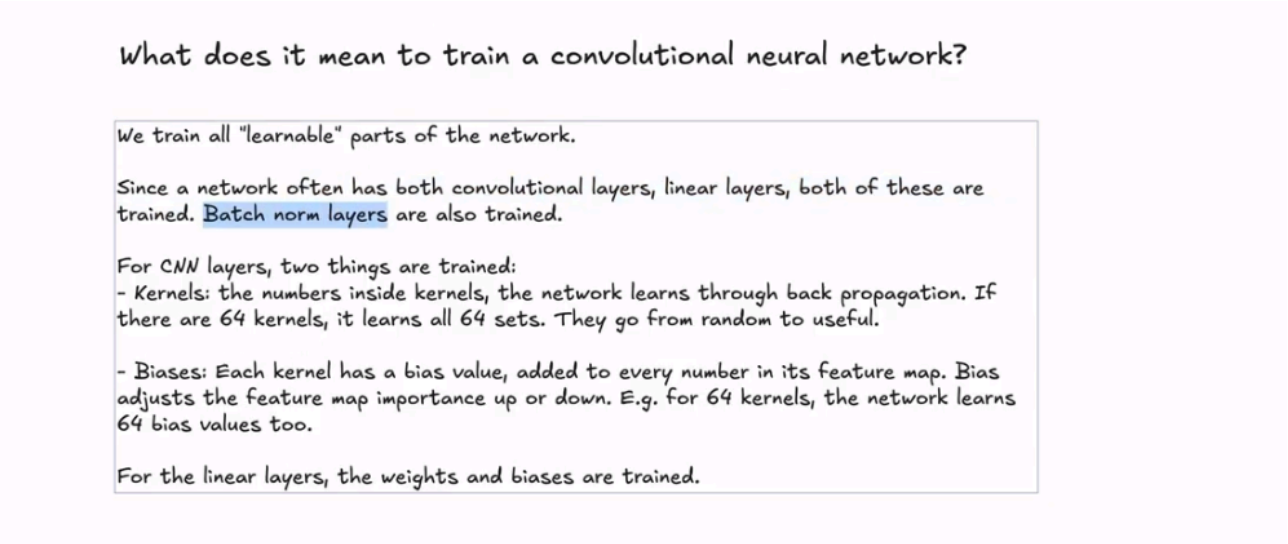
Batch Normalization – output of one layer is reset to predictable range for subsequent layers say L2 used a specific output from L1, **Batch Norm** (a layer in itself), through **back propagation,** learn how good stable output from L1 should be by adjusting **up & down** without L2 seeing from outputs. Hence faster training as L2 won't **'shoot for a moving target'** each time.



When training a network**,** it trains **"learnable"** parts of the network. For **CNN layers,** 2 things are trained:

→ **Kernels:** here network learns through back propagation eg 76 kernels it learns 76 sets (they go from random to useful)

→ **Biases:** Each Kernel has bias value, added to every number in feature map, whose (F. map) importance is adjusted up / down



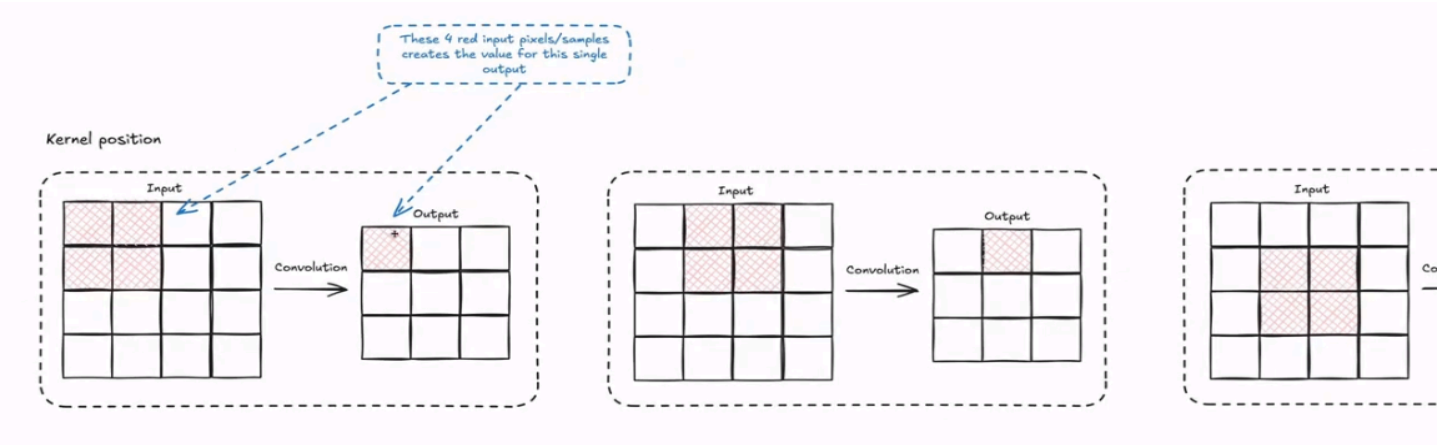
CNN Hyperparameters

These are set so that the different layers in the network when trained each can extract different things basically, which are:

→ **Kernel Size**

→ **Stride -** how many pixels each of these patches jump each time one patches a new statement when the kernels slide over a new matrix.

→ **Padding –** allows us capture the edges of input matrix accurately

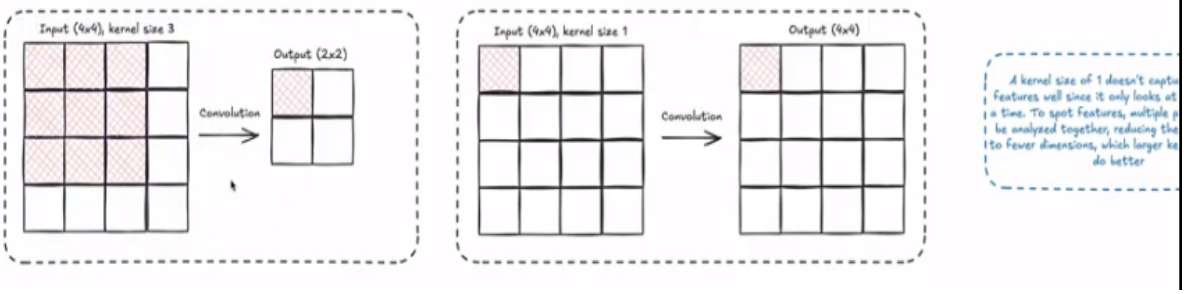


Each of the patches will then produce a pixel of its own in the output matrix.

1. Kernel Size

These are specialized for handling different tasks where big kernels capture broader simple patterns which gathers quick general understanding of input before more details but this is expensive computationally as each of the parameters will be trained in the kernel and if one have a big kernel each of the many params have to be trained.

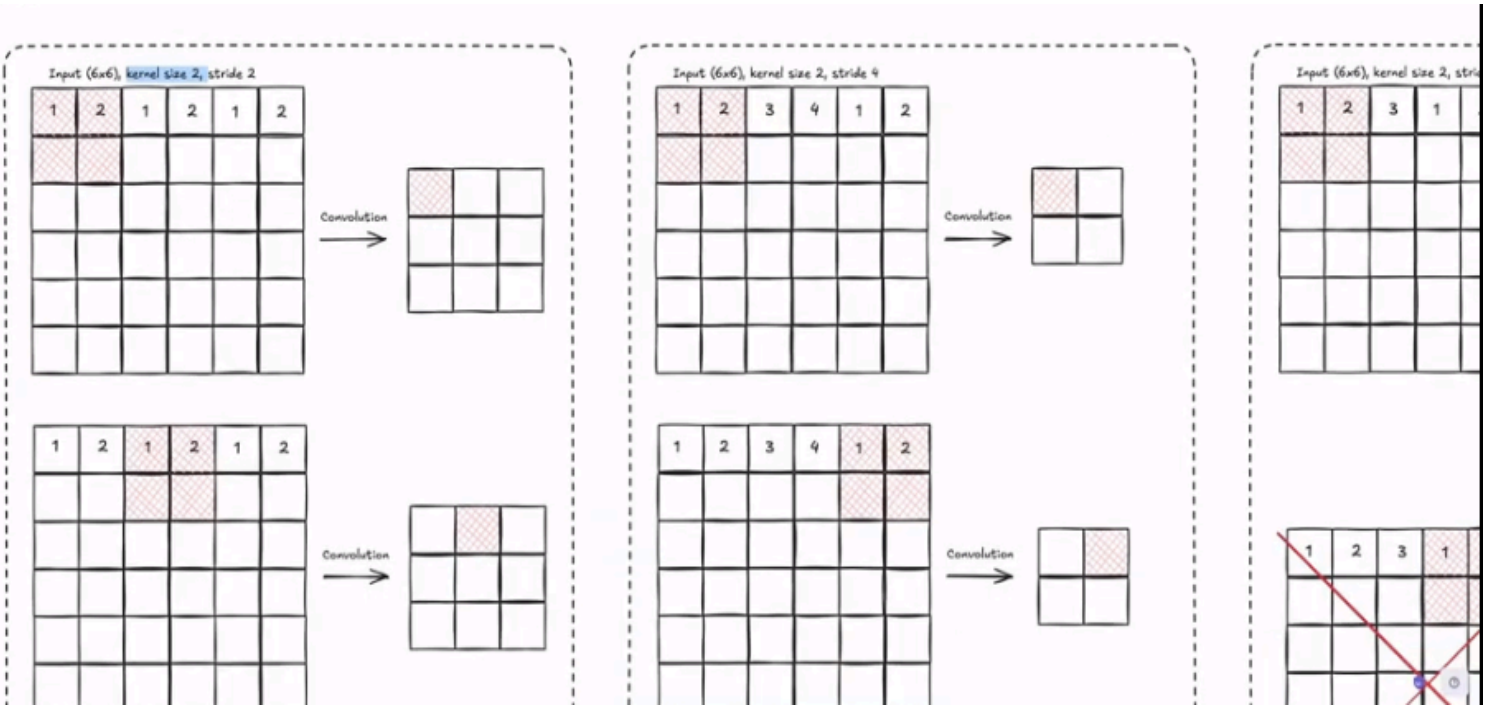
For a smaller kernel, there's smaller local details that are captured hence the network learns better which adds more **ReLU (activation steps)** in between, easier learning of complex patterns from the same image area and fewer parameters too.

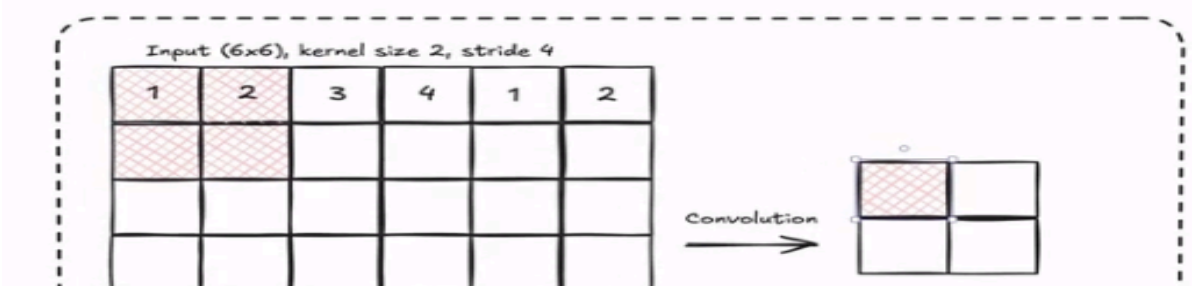


2. Stride

Is how many pixels in the kernel skips over the input in CNN affecting how much one learns and how fast it runs, balancing detail captured and computational efficiency. Imagine one has a high(2 or 3) stride. We save computational energy and work faster but skip details, jump more and shrink output. A small stride (1 stride) moves one pixel, grabbing more details making a bigger output

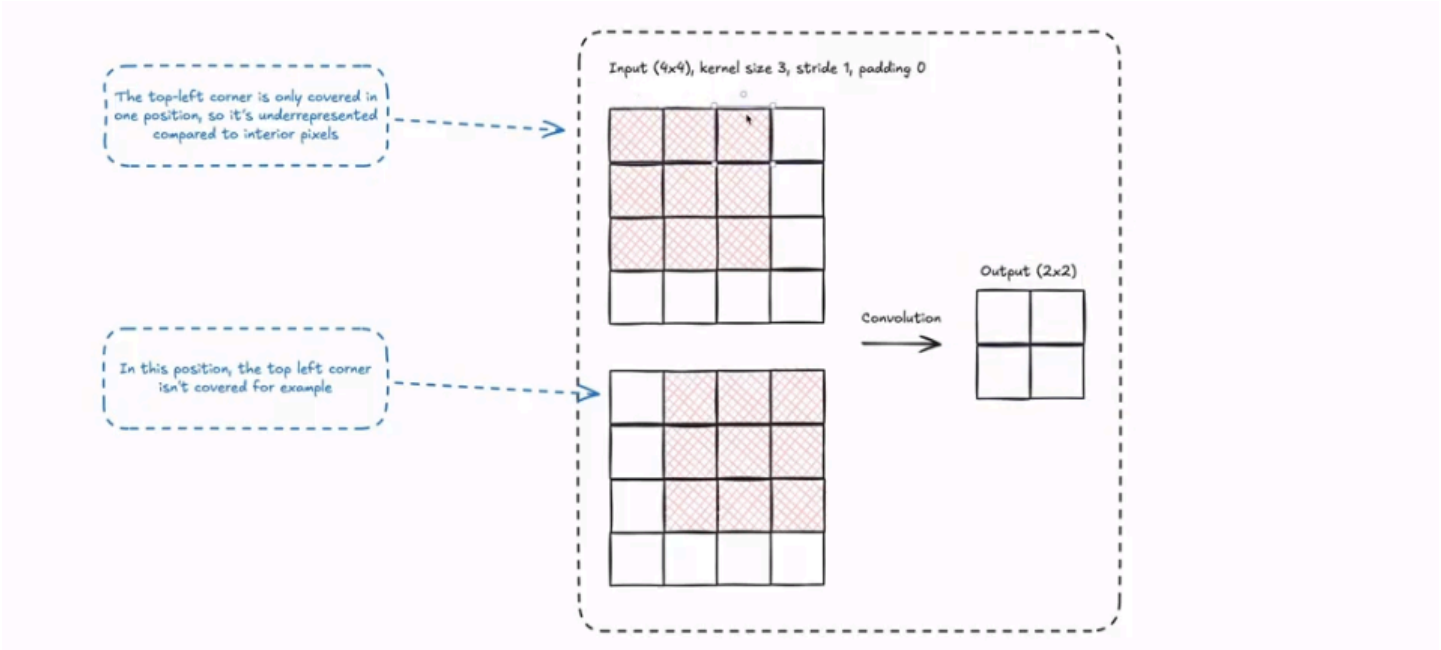
To determine allowed strides one needs to ensure the kernel can **slide across input** without leaving **uneven gaps.** The output size must be a whole number and can be calculated using → **(input size -Kernel size + 2 * padding) / stride + 1,** and the result should be an integer.

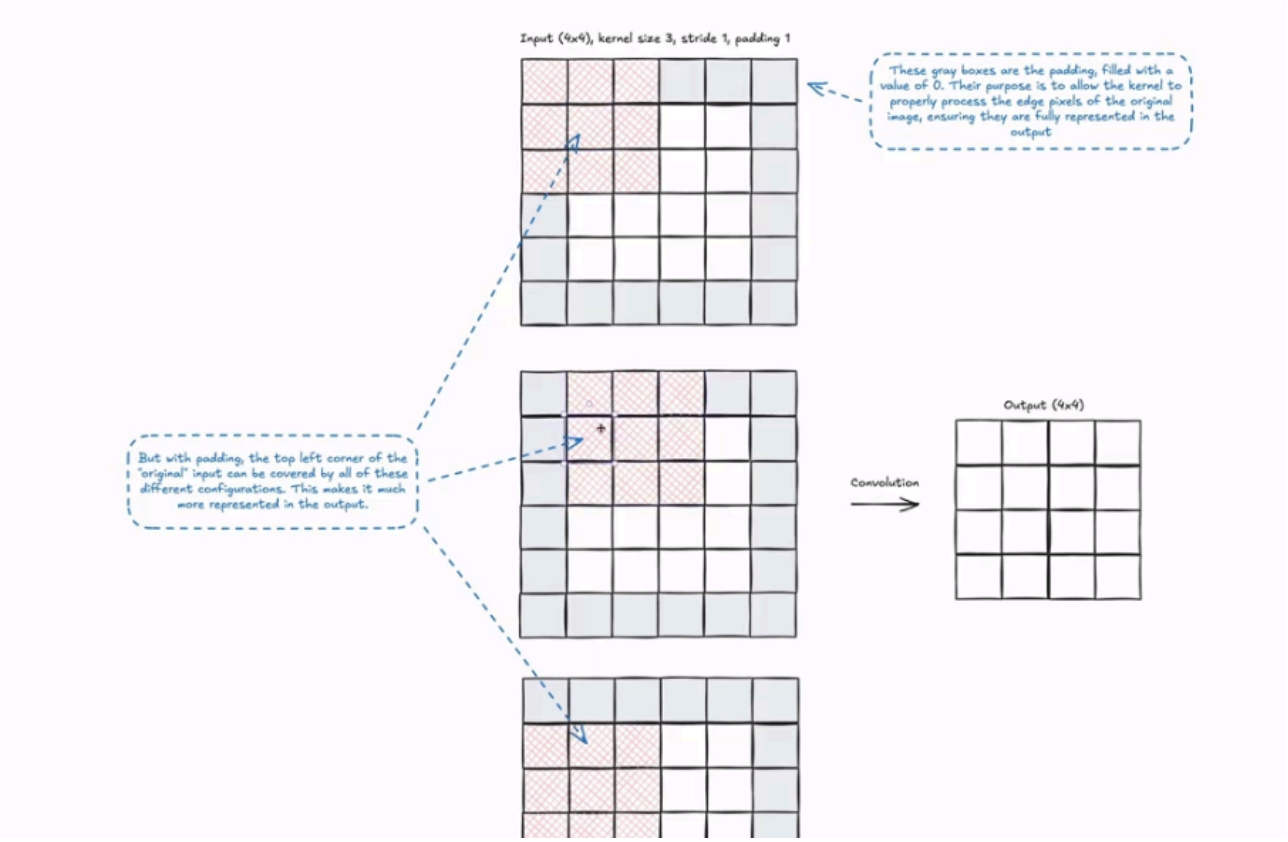




3. Padding

Outputs can shrink with each layer losing edge information. Padding maintains or adjusts the size of the output matrix. If say we have input matrix **(3 * 3),** and kernel size of 3 it will create a 1 * 1 output matrix, a heavy damage in reduction. With padding sliding the 3 *3 matrix with **padding 1 & stride 1,** leaves a **3 * 3 output matrix** instead of 1 * 1. This way the adding can be used to help and determine control of the actual output matrix size we get. These padding values are typically filled with 0s to help preserve edge pixels. In the example below the top-left corner is only covered in one position compared to middle pixels hence it becomes underrepresented and loses a high degree of information so we have few convolutions to help uncover details from these pixels unlike the middle pixels as more strides will include more info about the pixels.

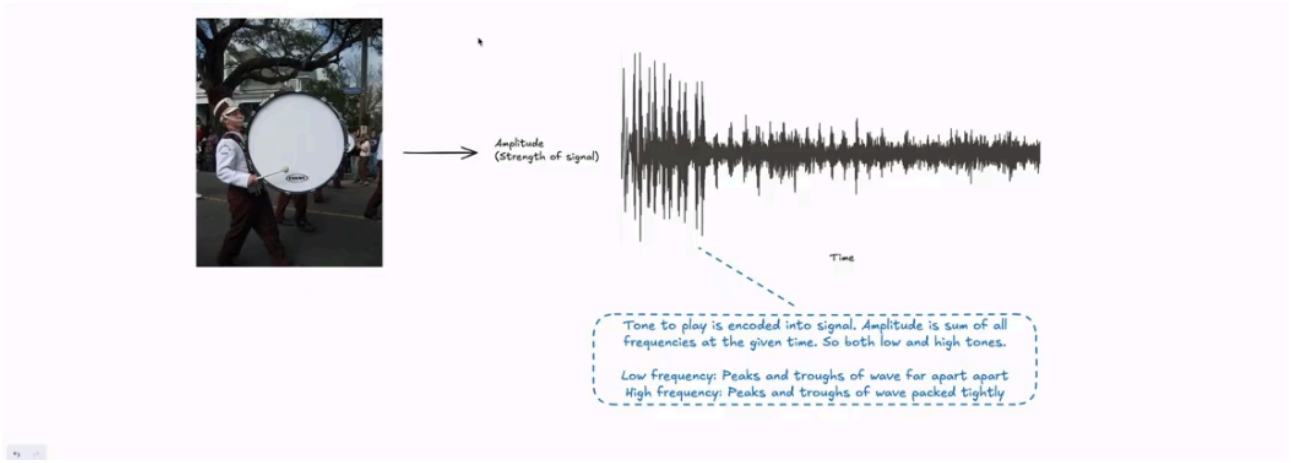




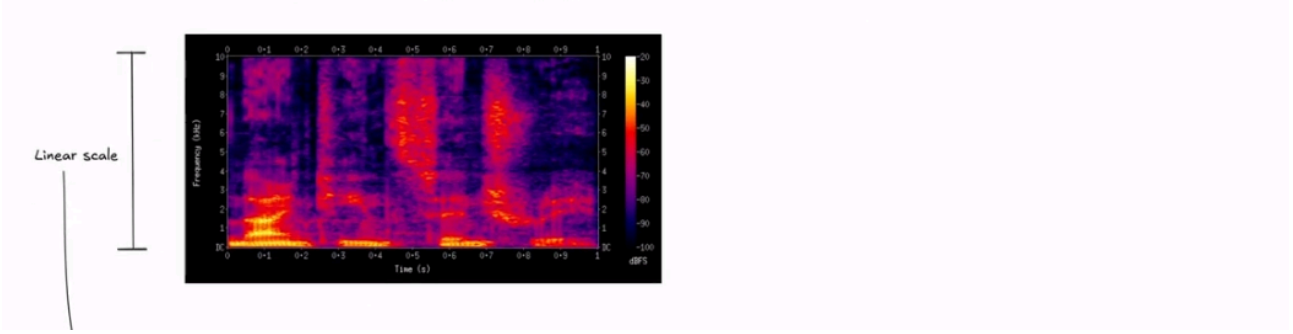
Audio and CNNs

Whenever one records an audio in computers they can be represented in the form of waveform. **Analog** is a continuous signal (normal sound of drum or real world sound). For **Digital,** we take a bunch of measurements in time-frame and get a bunch of discrete values out instead of continuous ones e.g. microphones hear audio and every millisecond takes measurement of how loud audio is which we can play in the computer.

Sound rate is the rate of measurements of the microphone in Hz. And the output of all these is stored in .WAV files. The figure below puts the microphone near the drum and listens ending up with a .WAV file that will measure **amplitude** (strength of signal – Y-axis). Tone to play is encoded into the signal



But we need to convert it into something we can use with CNN by converting a .WAV file into a **Spectrogram** using **Fourier Transform** to extract frequencies and their strengths where we can see time, frequency and frequency strength not the sum of all frequencies. They can be changed to match how humans perceive audio **Mel Spectrogram (** converted spectrogram **) - Y-axis** follows **Mel scale (** Bands of energy at bottom look taller and more spread out hence more recognizable by CNN, while top patterns look compressed and loose more details **)** this is due to human hears easily changes in low frequencies (100 – 200 Hz) than difference between high frequencies (10KHz)



The Mel-Spectrogram

We utilize the **Mel-Spectrogram** as our input feature. This transformation involves:

1. Short-Time Fourier Transform (STFT): Decomposing the signal into frequency components over short overlapping windows.

2. Mel-Scaling: Mapping frequencies to the Mel scale, which mimics human auditory perception by allocating higher resolution to lower frequencies. This effectively "images" the sound, where: * **X-axis**: Time

* **Y-axis**: Frequency (Log/Mel scale)

* **Pixel Intensity**: Amplitude (in Decibels)

This 2D representation allows 2D CNN kernels to learn:

* **Horizontal filters**: Temporal patterns (rhythms, onsets).

* **Vertical filters**: Spectral patterns (harmonics, pitch).

The **Mel spectrogram** is what will be fed into CNNs because:

1. it details each frequency ie specific base values for specific time-stamp (more detailed) by turning 1D line into 2D picture that CNN can look at compared to a point in waveform which is usually sum of all frequencies at that moment ie a **blended smoothie** while Mel a **deconstructed smoothie** giving CNN a better basis to know the signal.
2. **Mel Scale:** here the low Hz are more important than the high as it emphasizes Low frequencies by giving them more resolution while grouping high frequencies together.
3. **CNNs** better handling sounds shifted in time with **spectrograms** than **waveforms**

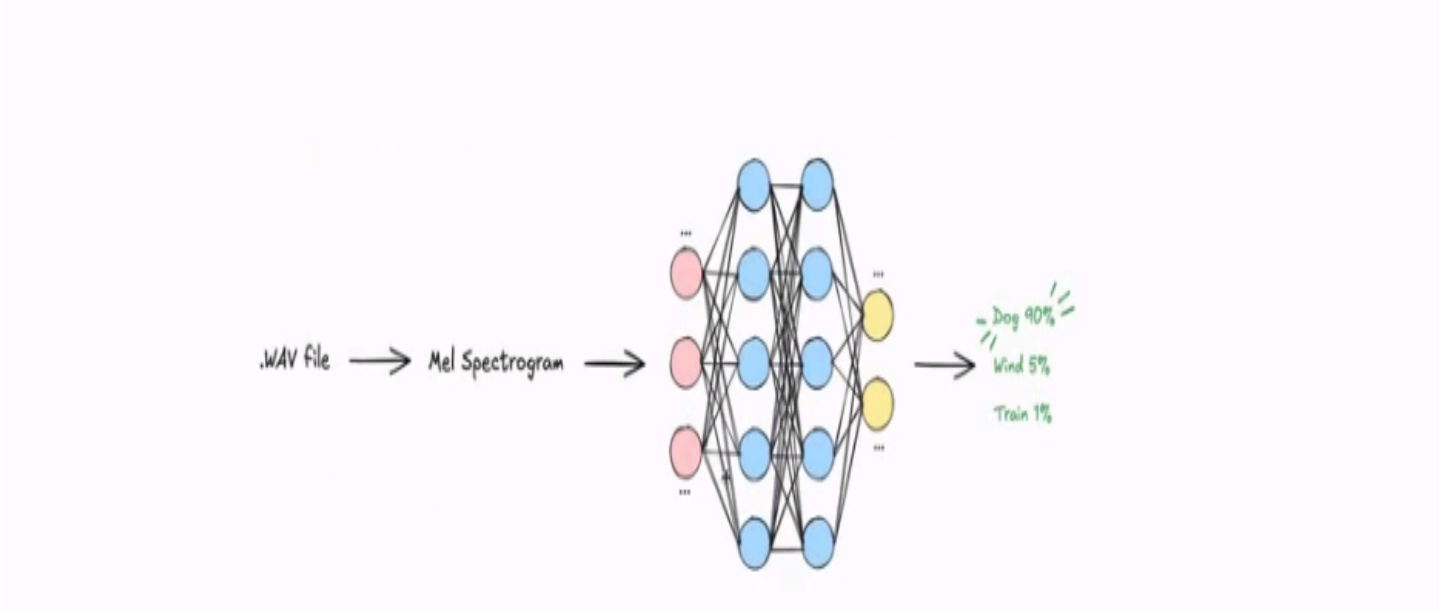
Sound Shifting and how it is seen by **CNN**:

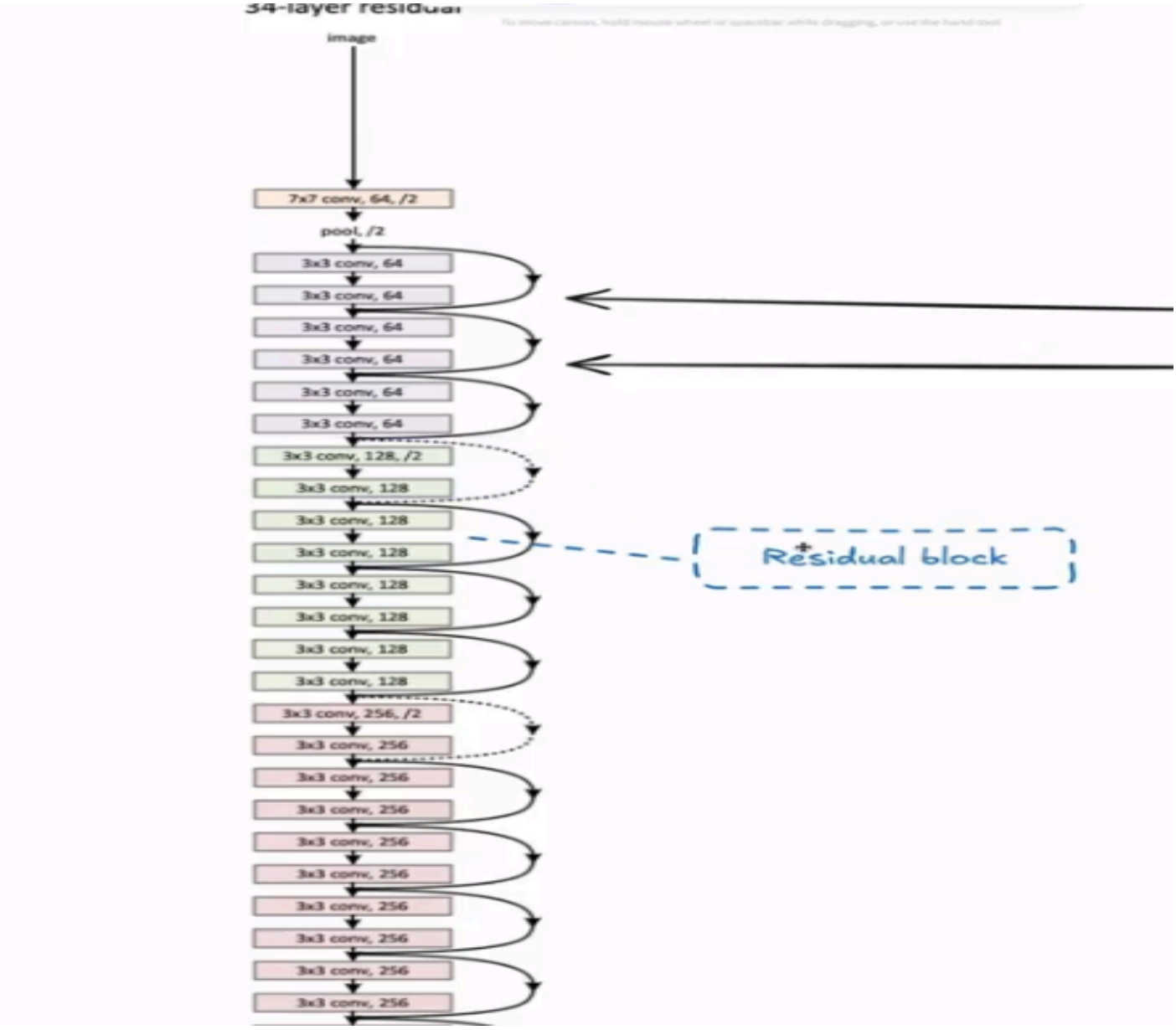
1. **Waveform:** shifting the signal in time completely changes raw input values making CNN learn same feature at multiple positions hardening generalization eg if say the initialized wanted sound frequency input was at the beginning of the audio, CNN then learns this pattern but when the target sound shifts to a different position of the audio file but same sound the network returns unwanted response (losses)
2. **Mel Spectrogram:** time shift moves pattern horizontally but overall pattern stays intact. CNNs can detect this shape at different time positions making them more time-invariant.

This was the concept of translational invariance we had talked about earlier.

Model Architecture

Data model sample – **ESC-50. ResNet** will be the model research paper which we will base our architecture on. It has a **Deep Network, 7 * 7 CNN** with multiple CNN classified into blocks of 2 **Residual Blocks,** each having different kernel sizes then all these are put into an **average pool** and **linear layer** used to classify the network.





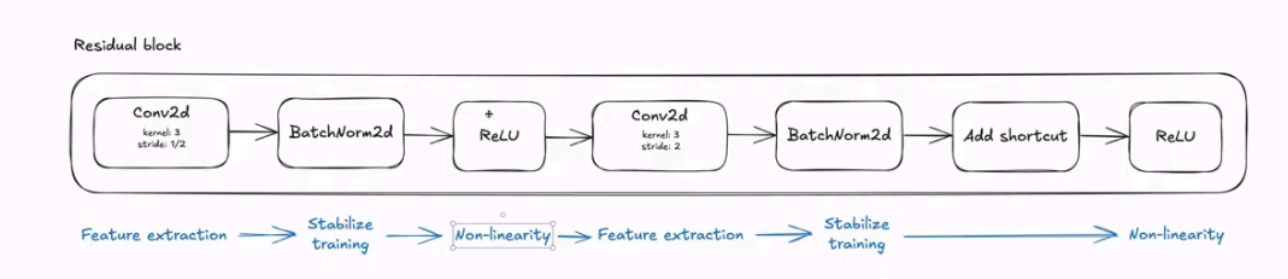
One of the things introduced in the paper was **Shortcuts** due to **vanishing gradient problems** in a Deep CNN. This is because gradients get smaller and smaller hence earlier layers stop learning. When forward-pass training one gets a loss all the way down to the average pool and calculates the gradient of each layer which tells the previous layer these **weights and biases** need to be changed in a way to minimize loss in that layer. This correction problem is passed all the way up to the initial layer and because it has to pass through all the layers it might get pretty weak at the beginning hence **Vanishing Gradient Problem** hence Ws and Bs won't be changed a lot or learn a lot hence Network won't perform very well.

Shortcuts were introduced to solve this problem. They allow gradients flow backwards more directly i.e. input of a block added directly to **output** of that same block's convolutional layers in that instead of adding whatever output is generated directly it will be the whatever generated output and the initial immediate previous input into the block. **Without Shortcut,** a normal layer has to learn the entire **final output** from scratch but

with shortcut existing layer will only improve existing feature map by making a difference within the feature map hence learning less

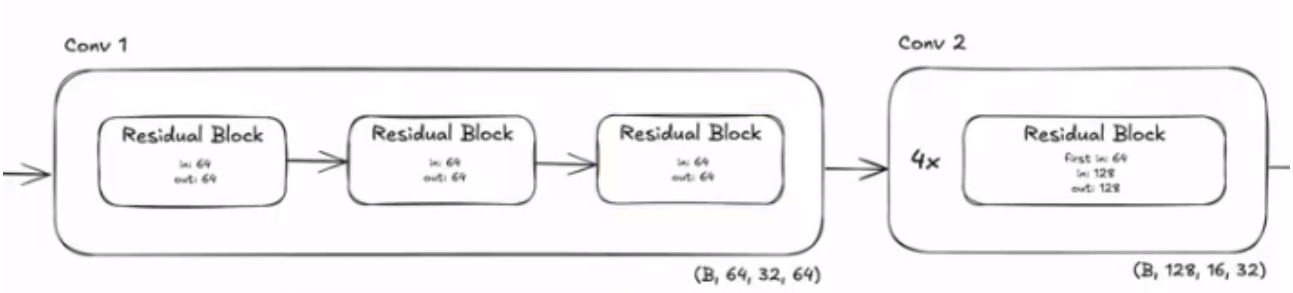
Also during **back-propagation,** the addition operator acts as gradient **distributor,** copying the **incoming error signal** and sending it down both paths simultaneously. Shortcut path is an **uninterrupted highway** for the signal. Even if gradient through main layers (Path 1, **F(x)**) weakens, clean gradient from shortcut (Path 2, **x**) ensures strong signal always reaches earlier layers, preventing stalled training.

Residual blocks are sets of Convolutional layers put into blocks where one can add input of layer to output of layer to create shortcuts for proper training. **BatchNorm2d** in between decouples the 2 convolution blocks where the **ReLU** adds nonlinearity

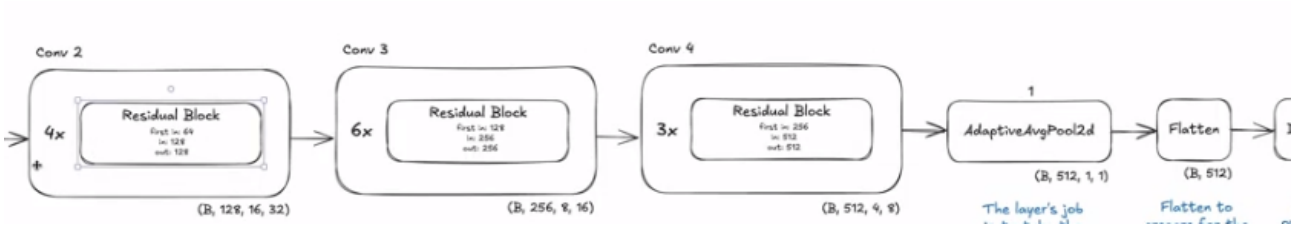


For the initial convolutional process, it will have a single layer will **kernel of 7, stride of 2, padding 3** then will have 64 output channels so 64 **feature maps** drawing broad features from ML spectrogram which will be passed through a **BatchNorm2d** to stabilize training then **ReLU function** then pull results to reduce dimensions. Initial dimensions (**gray spectrogram** is used as initial in this case **)** are **B** represents batch size, gray scale as 1 because of its gray spectrogram, 128 as **height** and frequency bins of spectrogram **,** 256 time-frame of spectrogram or width which produces **64 feature maps.** At **MaxPool2d,** we pass 64 channels because of the Feature maps and **32 height** → downsampled from 128, **64 width** → downsampled from 256.

Then the batch passed through the 1st **Convolutional Block** consisting of **3 residual Blocks (** because at the 34 Deep model we had **3 * 3** model **)**each with 64 feature maps. This deepens the network to learn more complex features without changing dimensions

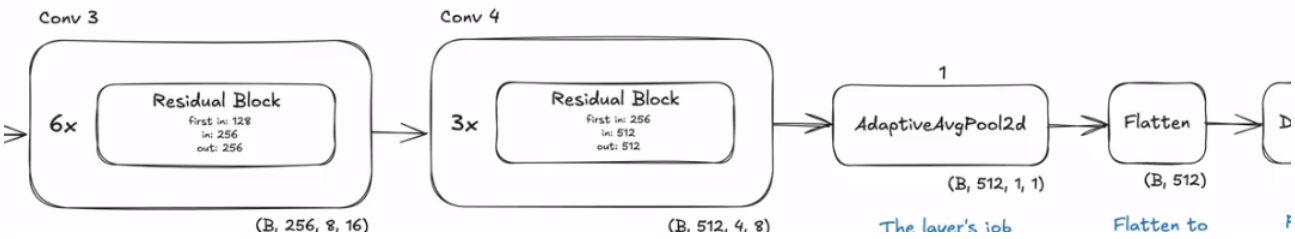


for the Conv 2, input channel is 64 (output of conv 1), **128** feature maps. Here **spatial dimensions** reduces while increasing number of feature channels to capture more intricate patterns

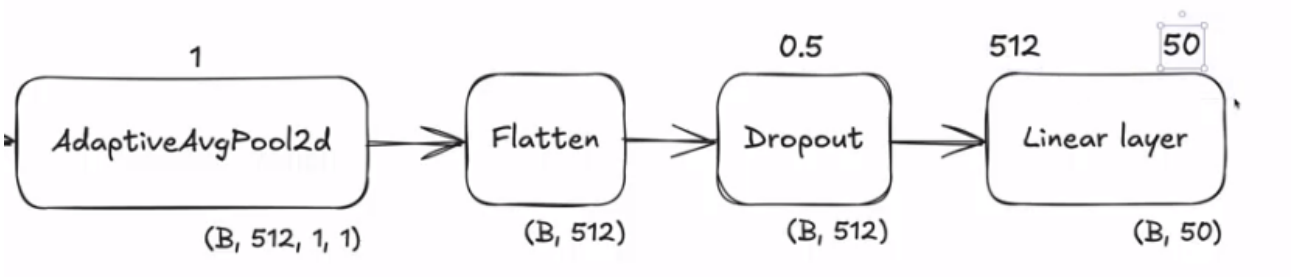


For **Conv 3,** we have 6 residual blocks. The 1st input channel will be output of **Conv 2 (128)** and output from each will be **256 feature maps (output channels)**and cutting the resolution by half again. **Conv 4** extracts final and most complex feature representation before the classification stage.

Adaptive Average Pool 2d role is take 4 * 8 **spatial grid** for each of the 512 channels and average it down to single 1 * 1 value. That will summarize the spatial information for each feature channel. **Flatten** is to prepare for the fully connected layer and flatten **Adaptive Average Pool 2d** into a single vector.

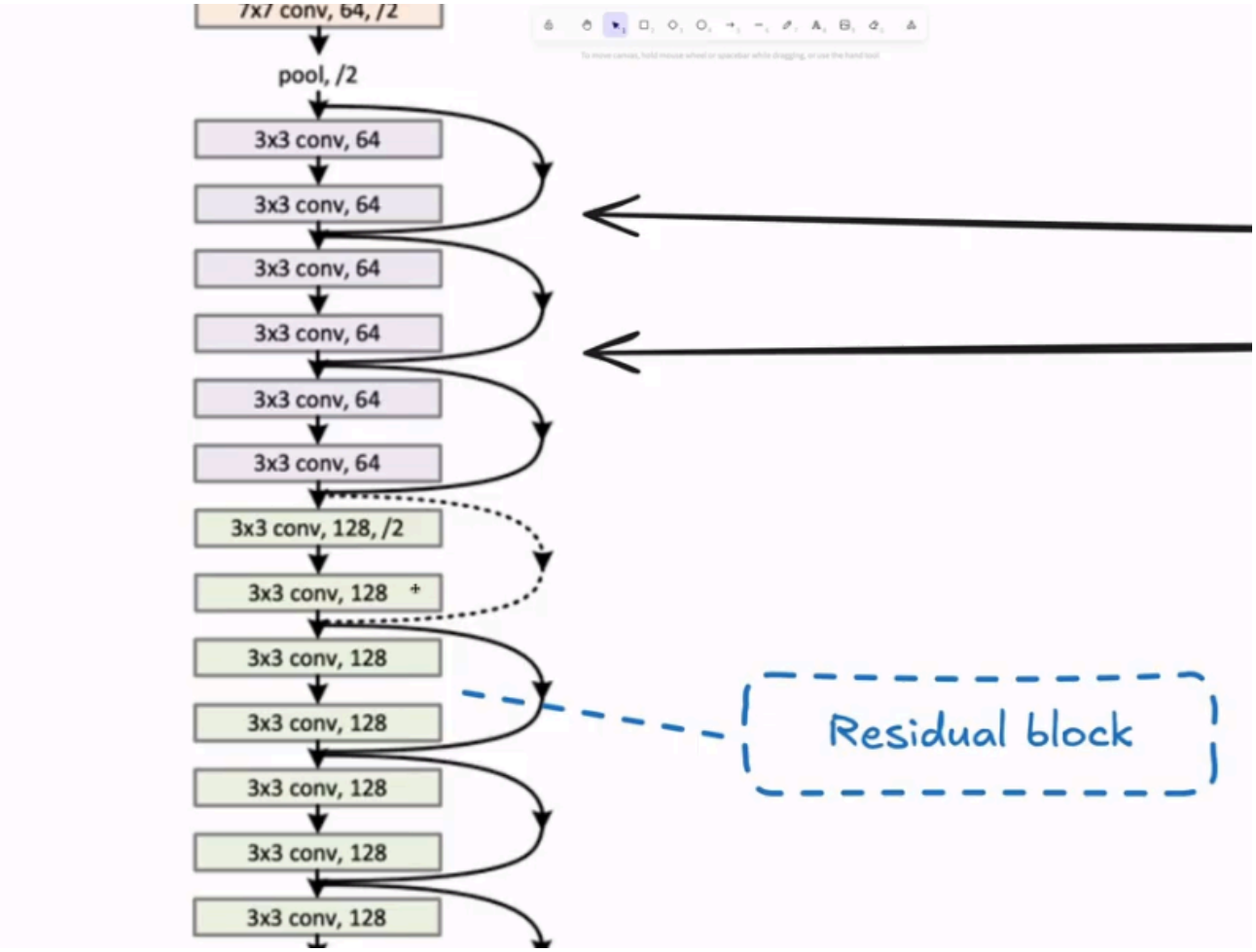


Dropout is to prevent overfitting during training to help the network not to be too overconfident by randomly skipping some neurons during training, preventing the network relying too much on specific neurons because sometimes they are switched off basically. The Dropout rate of 0.5 means about half of neurons drove off before shifting to the next layer. If all neurons aren't always available, then the networks don't always remember all the data and overfitting problem won't become prominent



Linear Layer's 50 corresponds to 50 different classes that exists in the dataset from the 512 input channels

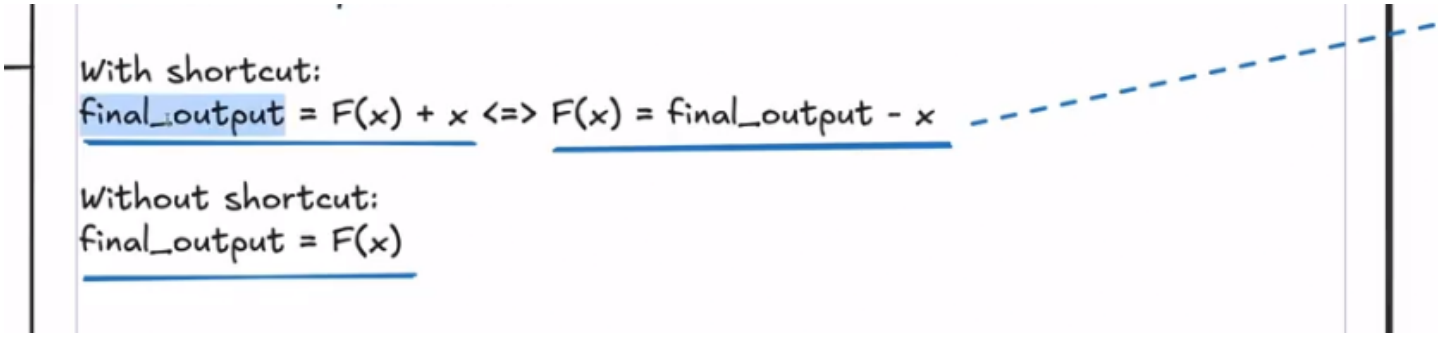
The **dotted lines** at the blocks is because the input at that layer isn't similar to output, hence a transformation is needed from input to output



Practical Implementation & System Architecture

Residual Connections

We employ the Residual architecture (ResNet). Instead of learning a direct mapping \mathcal{H}(x), the network learns a residual function **{F}(x) = {H}(x) - x**. The resulting mapping is **{F}(x) + x**.



This "shortcut" connection acts as an information highway:
1. Forward Pass: It allows identity mapping, meaning the network can easily pass information through without degradation if creating a complex transformation is unnecessary.
2. Backward Pass: It acts as a "gradient distributor," allowing gradients to flow directly to earlier layers, facilitating the training of very deep networks.

Implementation

ResNet-18 Backbone

Our model, AudioCNN, is a custom implementation of ResNet-18 tailored for

single-channel audio inputs.

Components

1. **Stem**: Initial 7 * 7 Convolution (stride 2) + MaxPool. This rapidly downsamples high-resolution spectrograms (1 * 128 * 256 to 64 * 32 * 64) to reduce computational load.
2. **Residual Stages**: Four stages of stacked ResidualBlocks.
    - **Stage 1**: 64 channels (3 blocks)
    - **Stage 2**: 128 channels (4 blocks)
    - **Stage 3**: 256 channels (6 blocks)
    - **Stage 4**: 512 channels (3 blocks)
3. **Classification Head**: Adaptive Average Pooling followed by a Linear layer mapping 512 features to the 50 ESC-50 classes.

Code Implementation: Residual Block

The block handles dimension matching via a 1 * 1 projection shortcut when strides reduce the spatial dimension.

```
class ResidualBlock(nn.Module):
def __init__(self, in_channels, out_channels, stride=1):
super().__init__()
# 3x3 Convolution with Batch Norm
self.conv1 = nn.Conv2d(in_channels, out_channels, 3, stride, padding=1,
bias=False)
self.bn1 = nn.BatchNorm2d(out_channels)
self.conv2 = nn.Conv2d(out_channels, out_channels, 3, stride, padding=1,
bias=False)
self.bn2 = nn.BatchNorm2d(out_channels)

# Projection Shortcut (1x1 Conv) for dimension matching
self.shortcut = nn.Sequential()
if stride != 1 or in_channels != out_channels:
self.shortcut = nn.Sequential(
nn.Conv2d(in_channels, out_channels, 1, stride=stride, bias=False),
nn.BatchNorm2d(out_channels)
)

def forward(self, x):
out = torch.relu(self.bn1(self.conv1(x)))
out = self.bn2(self.conv2(out))
out += self.shortcut(x) # The Identity Mapping
out = torch.relu(out)
return out
```

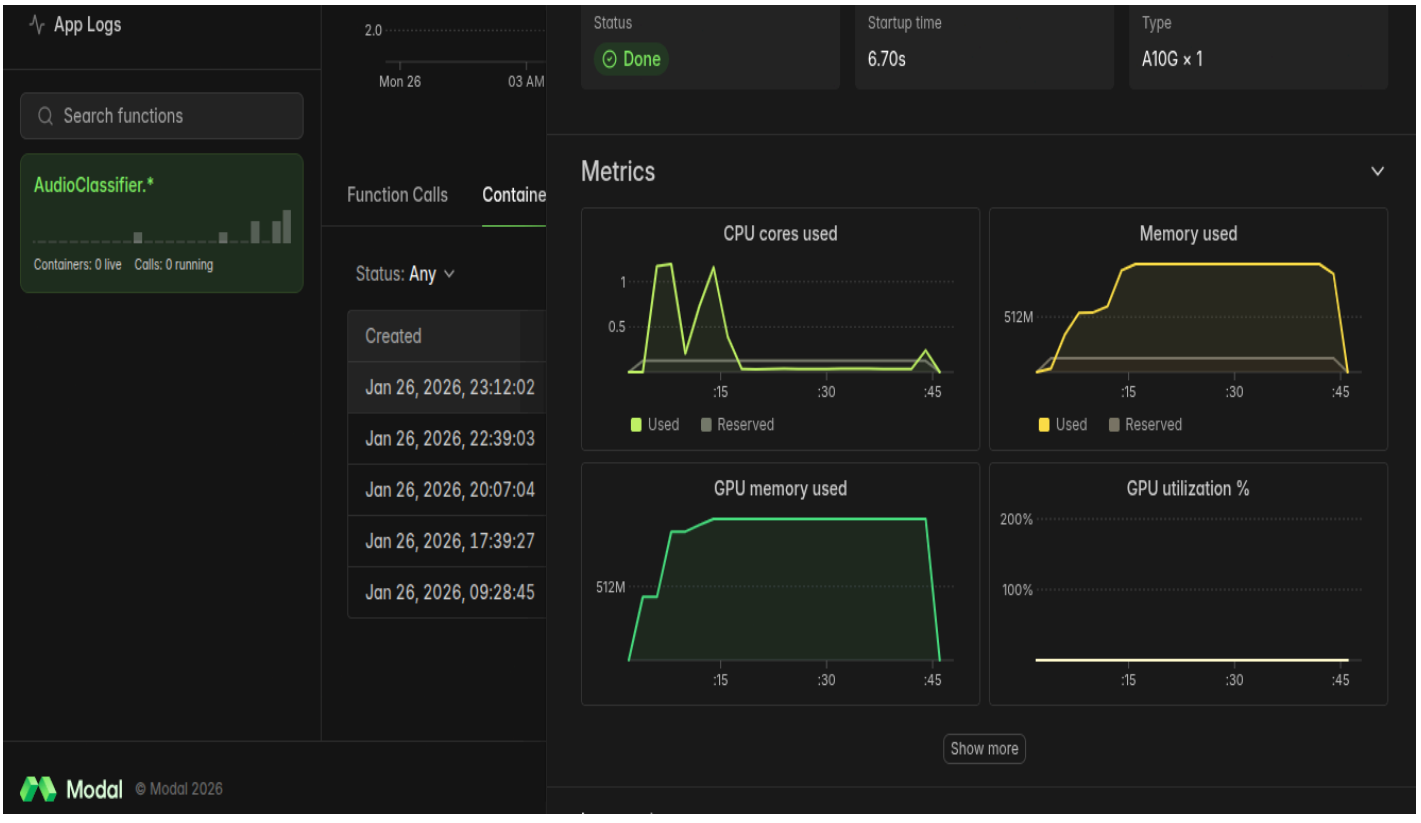
Cloud-Native Infrastructure (Modal)

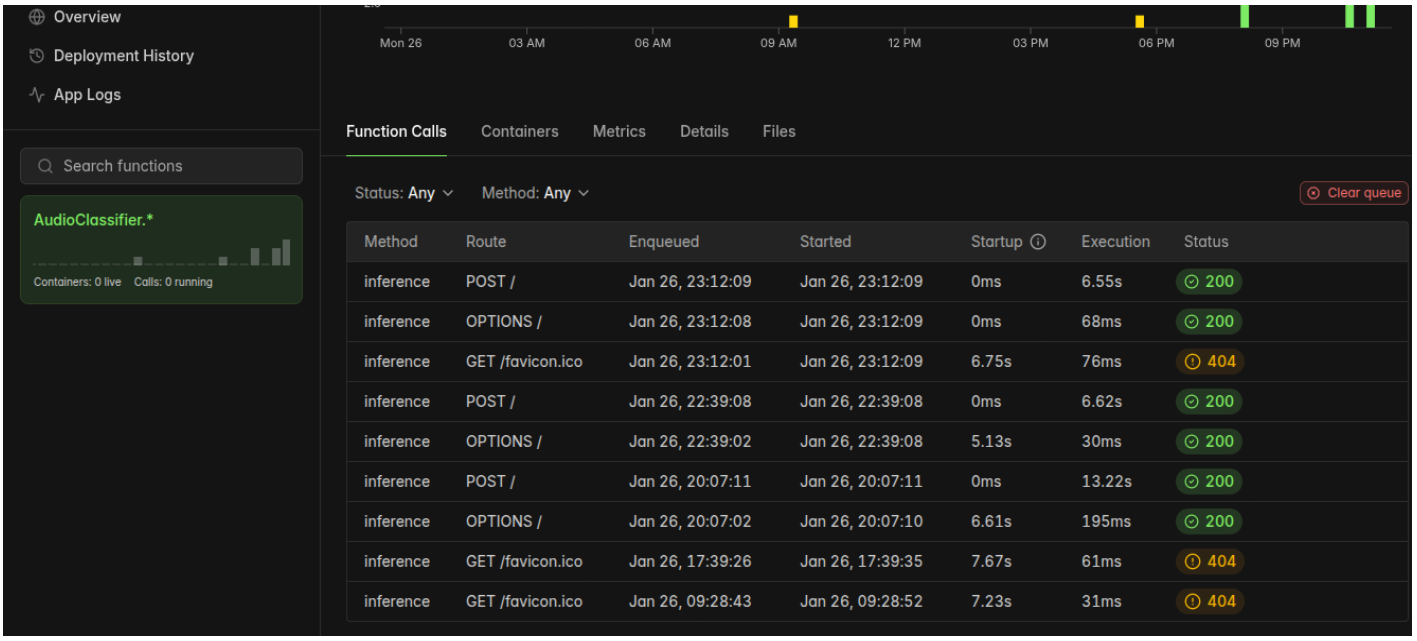
The system adopts a serverless architecture using **Modal**. This shifts infrastructure complexity into Python code.

- **Dependency Management**: Images are built programmatically, installing system libraries (libsndfile1, ffmpeg) and Python packages (torch, torchaudio).
- **Distributed Storage**: modal.Volume persists the dataset (/opt/esc50-data) and model artifacts (/models), ensuring high-throughput data access during training.
- **Ephemeral Execution**: GPU containers (NVIDIA A10G) are provisioned on-demand for training and scaled to zero when idle, optimizing cost.

Container Usage Over Time Function Call Hierarchy


Overview
Deployment History
App Logs
Search functions
AudioClassifier.*
Containers: 0 live Calls: 0 running
Function Calls
Containers
Metrics
Details
Files
Containers
1.0
500m
Mon 26
06 AM
12 PM
06 PM
Live
Total
CPU
40m
20m
Used
Reserved
Memory
41M
20M
Network
2/s
1/s
Egress
Ingress
GPUs
A10G
GPU memory
Memory

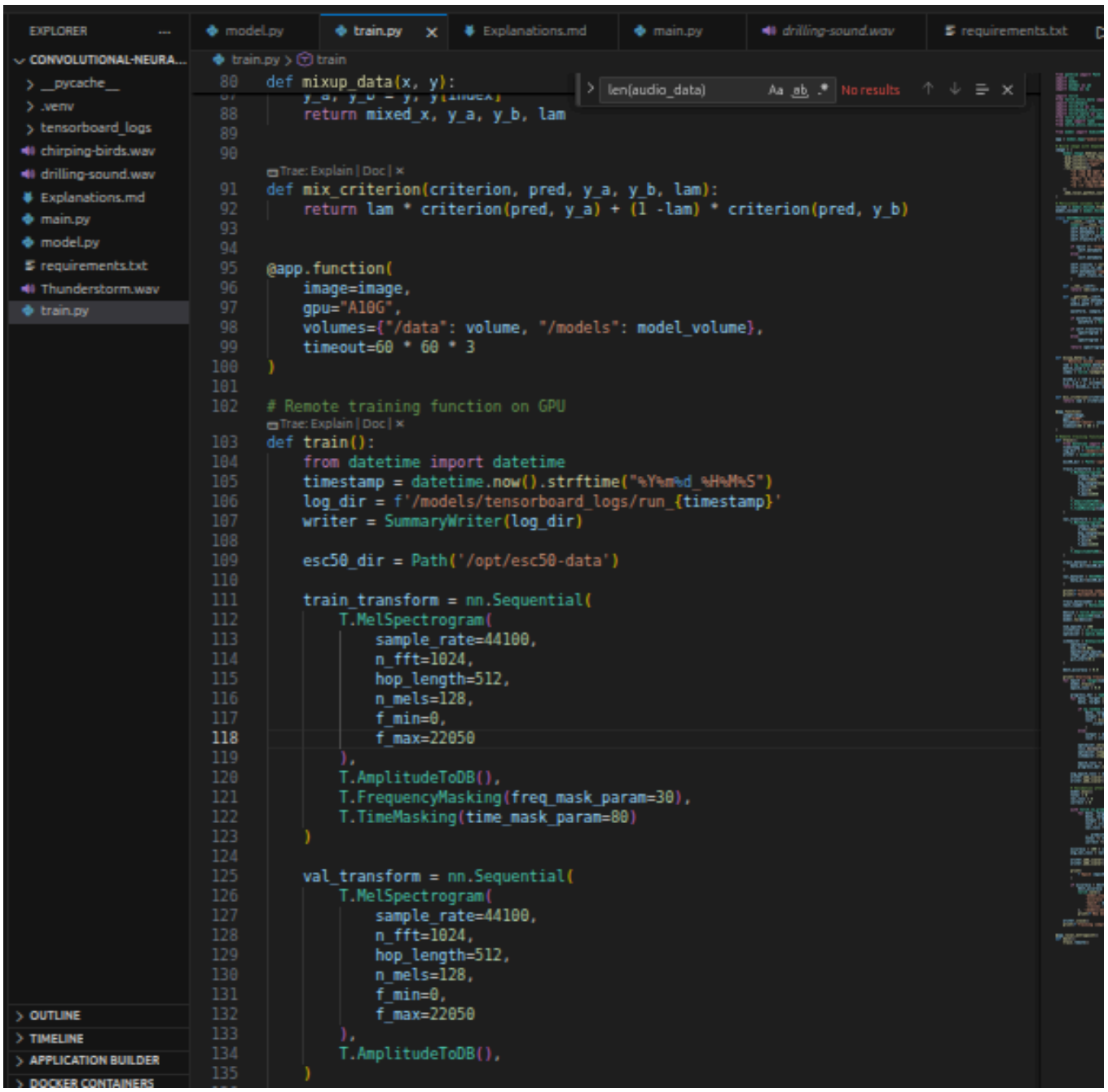

App Logs
Search functions
AudioClassifier.*
Containers: 0 live Calls: 0 running
2.0
Mon 26
03 AM
Function Calls
Containe
Status: Any
Created
Jan 26, 2026, 23:12:02
Jan 26, 2026, 22:39:03
Jan 26, 2026, 20:07:04
Jan 26, 2026, 17:39:27
Jan 26, 2026, 09:28:45
Status
Done
Startup time
6.70s
Type
A10G x 1
Metrics
CPU cores used
Memory used
512M
GPU memory used
GPU utilization %
200%
100%
:15
:30
:45
Used
Reserved
Show more
Modal
© Modal 2026



Signal Processing Pipeline

We employ a transformation pipeline to convert Time-Domain signals to Frequency-Domain representations.

* **Resampling**: standardized to 44.1kHz.
* **Mel-Spectrogram**:
* FFT Size: **2048**
* Hop Length: **512**
* Mel Bands: **128**
* Frequency Range: **0Hz - 22,050Hz** (Nyquist)
* **Log-Scaling**: AmplitudeToDB conversion.



Advanced Regularization

Given the limited size of ESC-50 (2000 samples), we implement aggressive regularization. The exact parameters configured in train.py are:

A. Mixup

Mixup constructs virtual training examples by convexly combining pairs of inputs and labels: [Equation: \tilde{x} = \lambda x_i + (1-\lambda)x_j]

* **Beta Distribution**: \alpha = 0.2 (np.random.beta(0.2, 0.2))
* **Probability**: Applied with **70% probability** per batch (if np.random.random() > 0.7).

B. SpecAugment

Instead of time-domain distortion, we apply masking directly on the spectrogram: * **Frequency Masking**: freq_mask_param=30 (Masks up to 30 Mel bands). * **Time Masking**: time_mask_param=80 (Masks up to 80 time steps).
This forces the network to rely on multiple feature correlations rather than specific frequency bands or transient identifiers.

Training Methodology

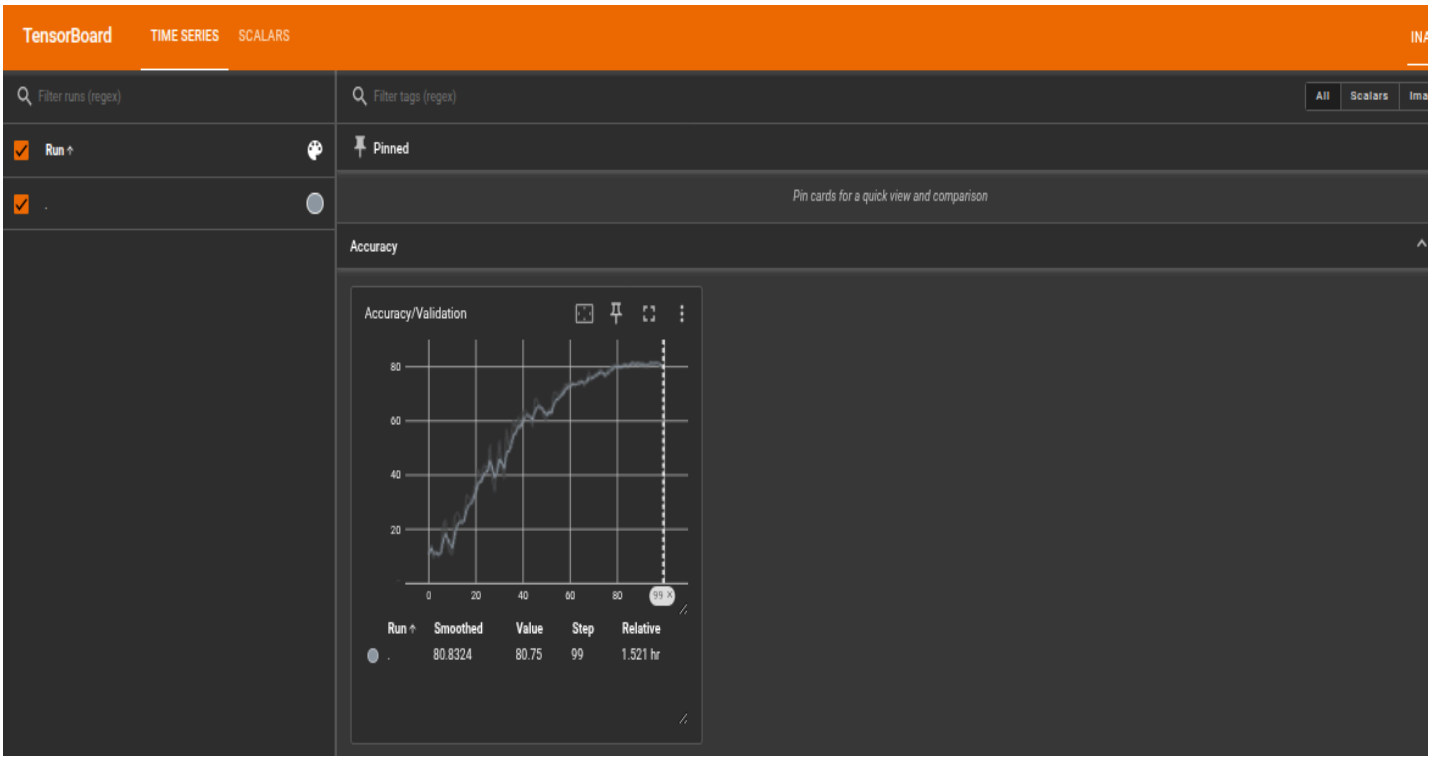
Optimization & Hyperparameters

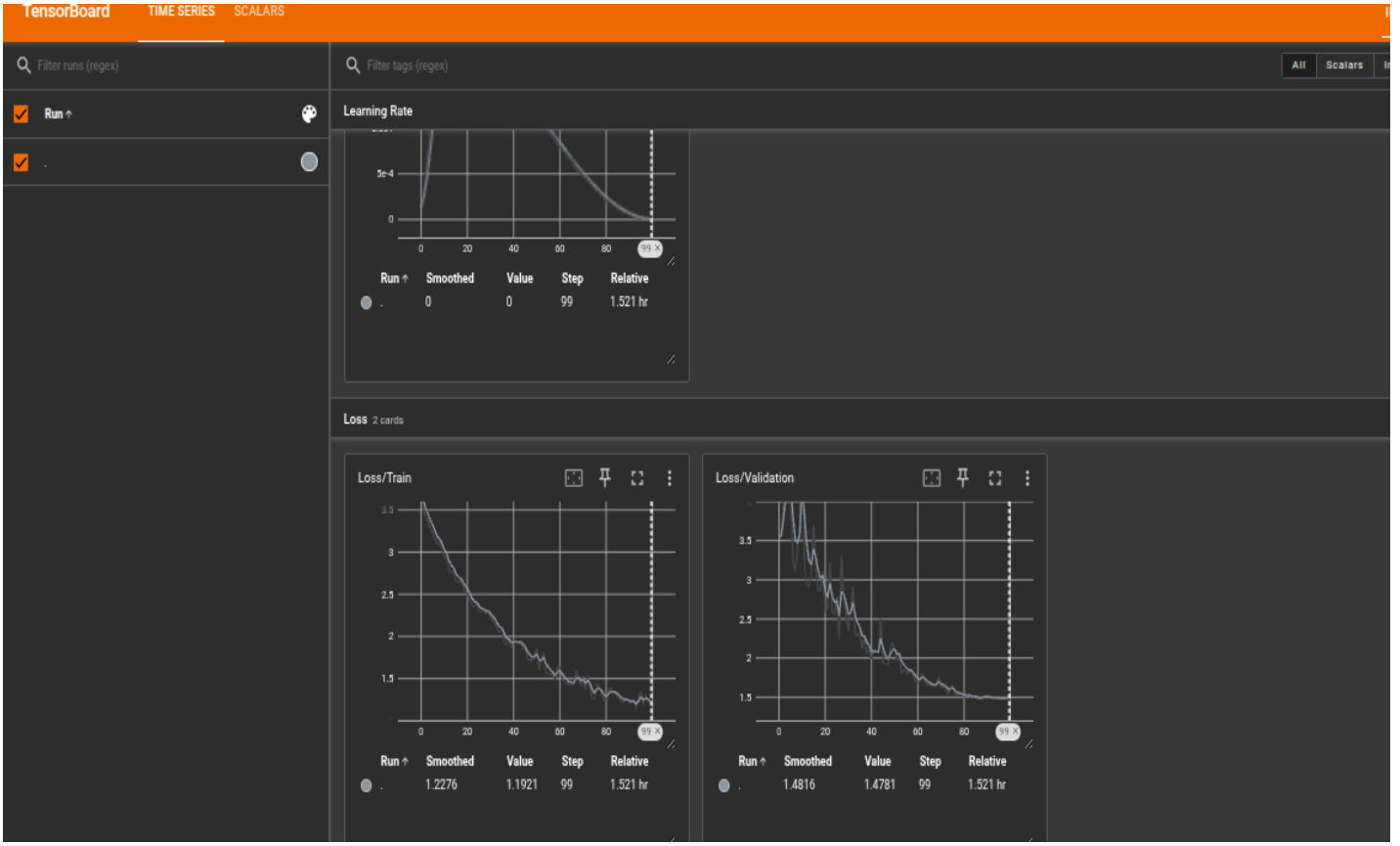
The training process is governed by the following verified hyperparameters extracted from the codebase (train.py):

- **Optimizer**: **AdamW**
  - weight_decay: 0.01 (decoupled regularization)
  - lr: Dynamic (see Scheduler)
- **Scheduler**: **OneCycleLR** (Super-convergence policy)
  - max_lr: **0.002**
  - pct_start: 0.1 (warmup for first 10% of training)
  - epochs: **100**
  - steps_per_epoch: Dynamic (based on len(train_dataloader))
- **Loss Function**: **CrossEntropyLoss**
  - label_smoothing: **0.1** (prevents model from becoming overconfident)
- **Batch Size**: **32**

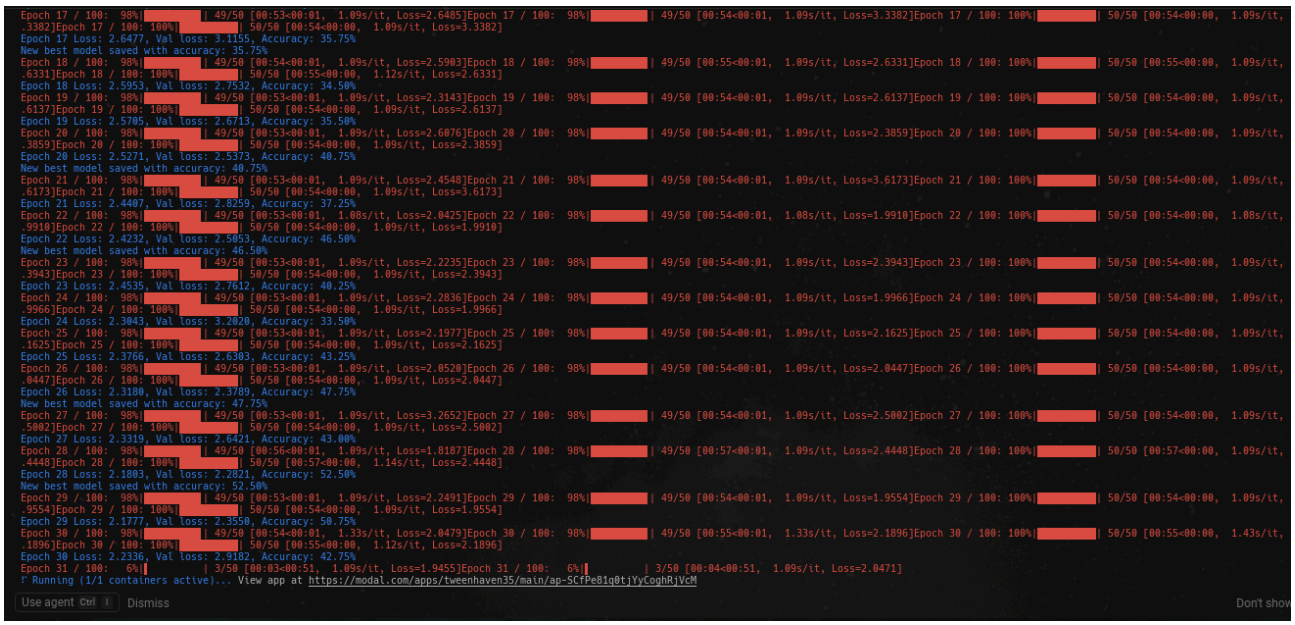
Performance Visualization

Training stability and validation accuracy are monitored via TensorBoard. The validation accuracy plateaus as the Cross-Entropy loss minimizes, triggering checkpoint saves for the best-performing model.





Figures 3: Training Loss (descending) and Validation Accuracy (ascending) over 100 epochs.

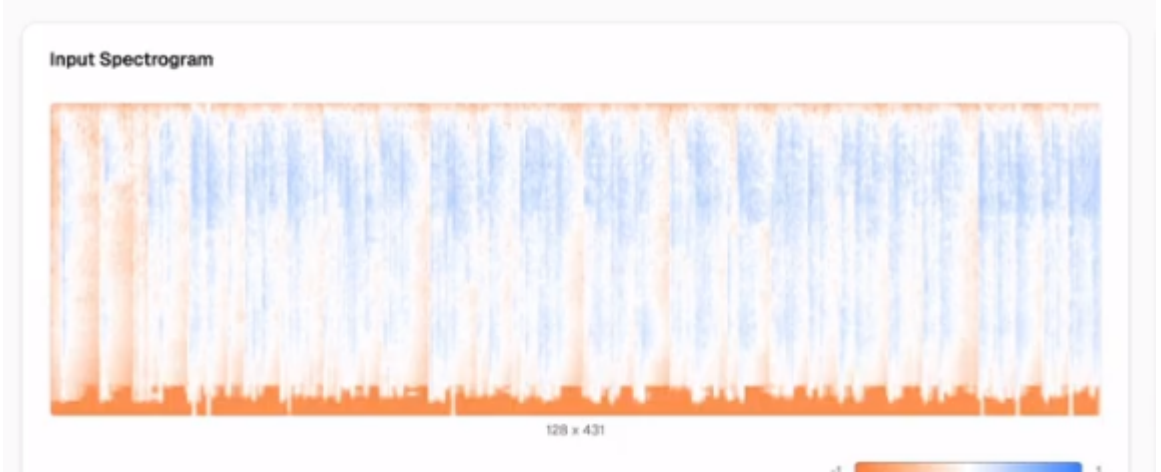


Inference & Explainability

Deployed as a specific AudioClassifier endpoint, the system includes an **Explainability Engine**. By hooking into ResidualBlock forward passes, we extract and visualize intermediate activation maps.

Feature Visualization

- **Input**: The raw waveform vs. the structured Mel-Spectrogram.
- **Activations**: The visuals below demonstrate how deep layers activate on specific semantic components of the audio, verifying that the model is learning relevant features (e.g., impact sounds, harmonic tails) rather than background noise.



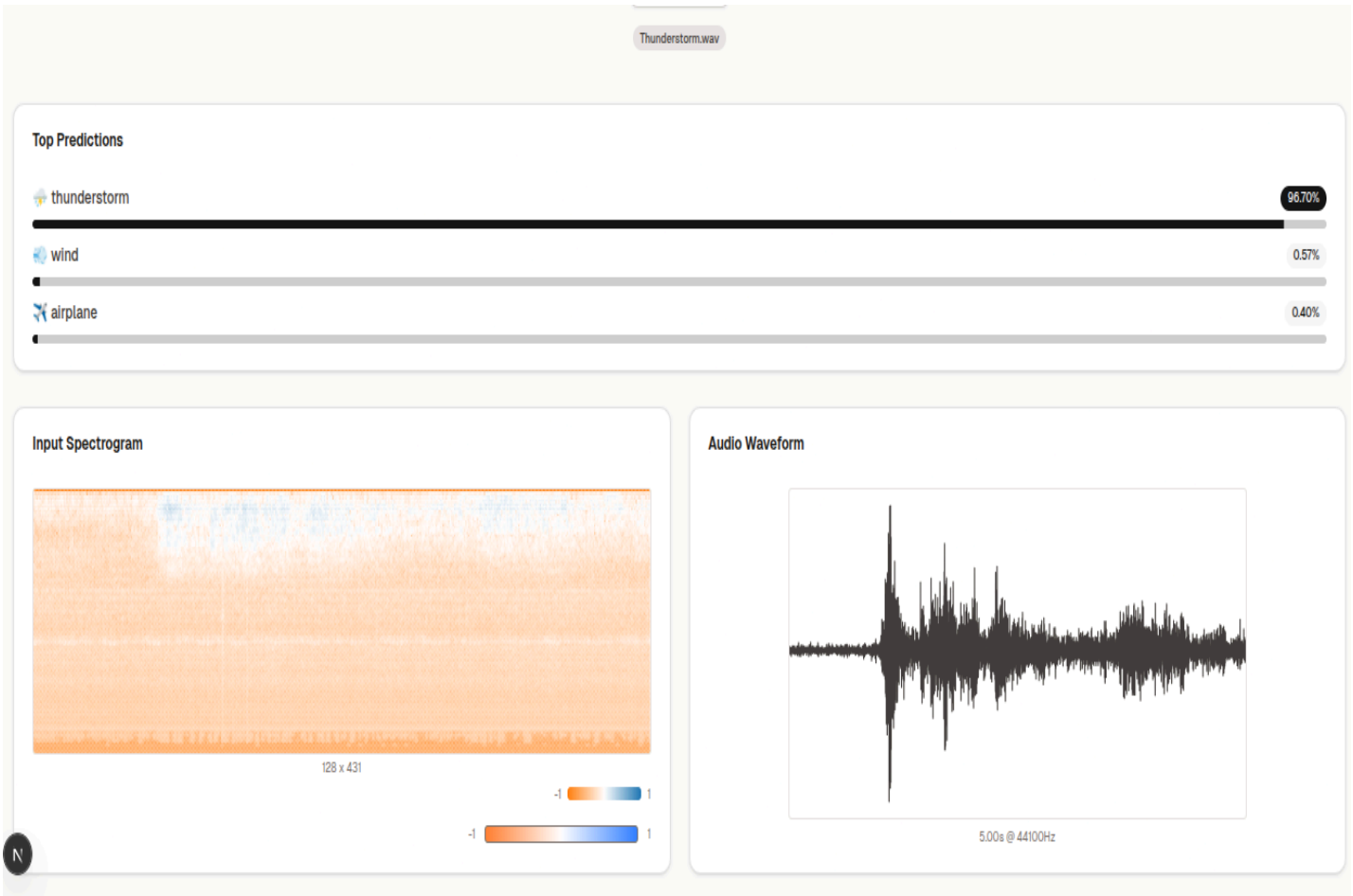


Figure 4: Raw Waveform (right) vs. Mel-Spectrogram Input (left)

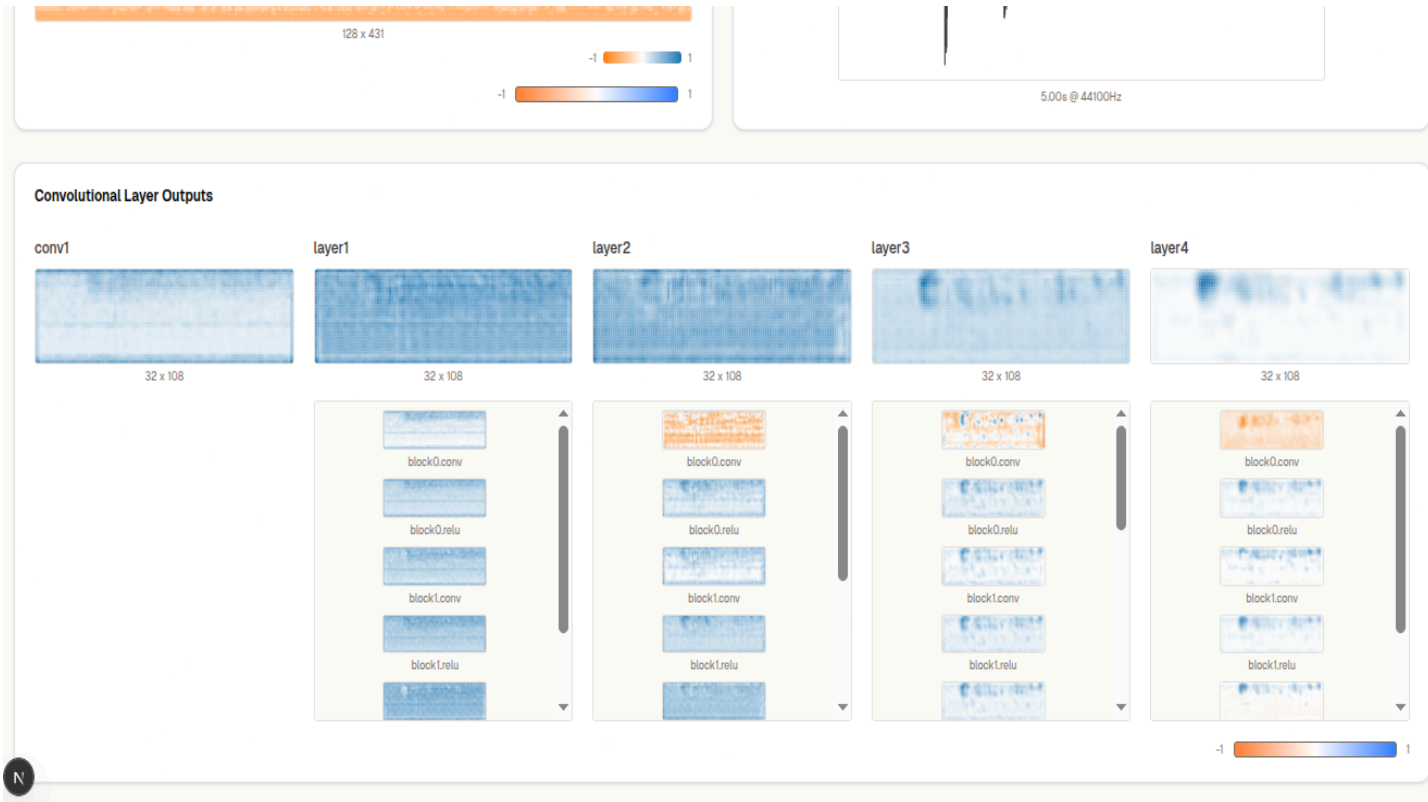


Figure 5: Visualization of activation maps from the final convolutional block.

Future Directions:

While this ResNet-18 implementation provides a robust baseline, the field of Audio Classification has continued to evolve. Recent research (2024-2025) highlights several advancements pushing the state-of-the-art (SOTA) on datasets like ESC-50.

1. Audio Spectrogram Transformers (AST)

Transformers, originally designed for NLP, have overtaken CNNs in many audio benchmarks. Models like **HTSAT-22** (Hierarchical Token-Semantic Audio Transformer) have achieved accuracies exceeding **98.25%** on ESC-50 [1]. These models utilize self-attention mechanisms to capture global dependencies across the entire spectrogram, unlike the local receptive fields of CNNs.

2. Two-Level Classification

A 2024 methodology proposes a **Two-Level Classification** system [5]. A Level 1 classifier determines broad categories (e.g., "Animals"), and specialized Level 2 classifiers make fine-grained distinctions (e.g., "Dog" vs. "Cat"). This hierarchical approach can reduce confusion between semantically distant classes.

3. Transfer Learning & Foundation Models

The current SOTA relies heavily on Transfer Learning from massive datasets like AudioSet or physically distinct domains (ImageNet). Foundation models pre-trained with

self-supervised learning (e.g., masked autoencoders for audio) are setting new benchmarks, suggesting that future iterations of this project should leverage pre-trained weights rather than training from scratch.

References

1. **Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun** (2016). Deep Residual Learning for Image Recognition. CVPR. arXiv:1512.03385
2. **Piczak, K. J.** (2015). ESC: Dataset for Environmental Sound Classification. Proceedings of the 23rd ACM international conference on Multimedia. GitHub Repository
3. **Hongyi Zhang, Moustapha Cisse, Yann N. Dauphin, David Lopez-Paz** (2018). mixup: Beyond Empirical Risk Minimization. ICLR. arXiv:1710.09412
4. **Daniel S. Park, William Chan, Yu Zhang, Chung-Cheng Chiu, Barret Zoph, Ekin D. Cubuk, Quoc V. Le** (2019). SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition. Interspeech. arXiv:1904.08779
5. **Recent ArXiv Submissions (2024)**: Methodologies on Hierarchical Classification and Transformer-based Audio analysis. arXiv:eess.A